VOL. 08

# Web3

Chains, contracts, and the decentralized apps you build on them

| | |
|---|---|
| Source | ethereum.org, soliditylang.org |
| Generated | 2026-10-10 |
| Level | Intermediate |
| Estimated study time | 25 hours |
| Language | English |
| Volume | Vol. 08 - Web3 |

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

# Table of contents

**How to use this volume . . . . . 7**

Chapter map . . . . . 7

**Chapter 1 - What Web3 is . . . . . 8**

1.1 Summary . . . . . 8

1.2 Concepts . . . . . 8

A chain of blocks . . . . . 8

Centralized versus decentralized bookkeeping . . . . . 9

Consensus without a boss . . . . . 9

Gas pays for shared resources . . . . . 9

1.3 Examples . . . . . 10

1.4 Common mistakes and best practices . . . . . 11

1.5 Practice exercises . . . . . 12

Solutions . . . . . 12

1.6 Mini test . . . . . 12

Answer key and explanations . . . . . 13

**Chapter 2 - Cryptography and accounts . . . . . 13**

2.1 Summary . . . . . 13

2.2 Concepts . . . . . 13

From key to address . . . . . 13

Two kinds of account . . . . . 13

Anatomy of a transaction . . . . . 14

Hierarchical deterministic wallets . . . . . 14

2.3 Examples . . . . . 14

2.4 Common mistakes and best practices . . . . . 16

2.5 Practice exercises . . . . . 16

Solutions . . . . . 16

2.6 Mini test . . . . . 17

Answer key and explanations . . . . . 17

**Chapter 3 - The Ethereum ecosystem . . . . . 17**

3.1 Summary . . . . . 17

3.2 Concepts . . . . . 18

What the EVM is . . . . . 18

Clients and nodes . . . 18
The networks you will meet . . . 18
The RPC surface . . . 18
3.3 Examples . . . 19
3.4 Common mistakes and best practices . . . 19
3.5 Practice exercises . . . 20
Solutions . . . 20
3.6 Mini test . . . 20
Answer key and explanations . . . 20
**Chapter 4 - Solidity basics . . . 21**
4.1 Summary . . . 21
4.2 Concepts . . . 21
The file frame . . . 21
Types worth memorizing . . . 21
Functions . . . 22
Events and errors . . . 22
4.3 Examples . . . 22
4.4 Common mistakes and best practices . . . 24
4.5 Practice exercises . . . 24
Solutions . . . 24
4.6 Mini test . . . 25
Answer key and explanations . . . 25
**Chapter 5 - Data structures and inheritance . . . 25**
5.1 Summary . . . 25
5.2 Concepts . . . 26
Structs and mappings together . . . 26
Data locations . . . 26
Modifiers . . . 26
Inheritance . . . 27
5.3 Examples . . . 27
5.4 Common mistakes and best practices . . . 32
5.5 Practice exercises . . . 32
Solutions . . . 33
5.6 Mini test . . . 33
Answer key and explanations . . . 33

**Chapter 6 - The toolchain . . . 33**
6.1 Summary . . . 33
6.2 Concepts . . . 34
The project shape . . . 34
The config file . . . 34
The three commands you use daily . . . 35
Tests with Node's runner . . . 35
6.3 Examples . . . 35
6.4 Common mistakes and best practices . . . 37
6.5 Practice exercises . . . 38
Solutions . . . 38
6.6 Mini test . . . 38
Answer key and explanations . . . 38
**Chapter 7 - Writing tokens . . . 39**
7.1 Summary . . . 39
7.2 Concepts . . . 39
The three standards side by side . . . 39
The allowance pattern . . . 40
Safe transfers and `safeTransferFrom` . . . 40
Build on OpenZeppelin . . . 40
7.3 Examples . . . 40
7.4 Common mistakes and best practices . . . 44
7.5 Practice exercises . . . 45
Solutions . . . 45
7.6 Mini test . . . 45
Answer key and explanations . . . 45
**Chapter 8 - Building a dApp frontend . . . 46**
8.1 Summary . . . 46
8.2 Concepts . . . 46
Two clients, two jobs . . . 46
The connection sequence . . . 46
The lifecycle of a write . . . 47
8.3 Examples . . . 47
8.4 Common mistakes and best practices . . . 51
8.5 Practice exercises . . . 51

Solutions . . . 52
8.6 Mini test . . . 52
Answer key and explanations . . . 52
**Chapter 9 - Reading blockchain data . . . 52**
9.1 Summary . . . 52
9.2 Concepts . . . 53
The anatomy of a log . . . 53
Two fetching styles . . . 53
What The Graph does . . . 53
What IPFS does . . . 54
9.3 Examples . . . 54
9.4 Common mistakes and best practices . . . 56
9.5 Practice exercises . . . 56
Solutions . . . 57
9.6 Mini test . . . 57
Answer key and explanations . . . 57
**Chapter 10 - Security, patterns, and upgradeability . . . 58**
10.1 Summary . . . 58
10.2 Concepts . . . 58
How the attack runs . . . 58
The vulnerability map . . . 58
Patterns that pay for themselves . . . 59
Proxies without magic . . . 59
10.3 Examples . . . 59
10.4 Common mistakes and best practices . . . 64
10.5 Practice exercises . . . 64
Solutions . . . 65
10.6 Mini test . . . 65
Answer key and explanations . . . 65
**Chapter 11 - DeFi and DAO building blocks . . . 66**
11.1 Summary . . . 66
11.2 Concepts . . . 66
Constant product . . . 66
Lending and liquidation . . . 66
Governance . . . 67

11.3 Examples . . . 67
11.4 Common mistakes and best practices . . . 72
11.5 Practice exercises . . . 73
Solutions . . . 73
11.6 Mini test . . . 73
Answer key and explanations . . . 74
**Chapter 12 - Testing, verification, deploying, and beyond Ethereum . . . 74**
12.1 Summary . . . 74
12.2 Concepts . . . 74
The four levels of testing . . . 74
What a good test file contains . . . 75
Before you deploy . . . 75
Verification and explorers . . . 75
Beyond the EVM . . . 75
12.3 Examples . . . 76
12.4 Common mistakes and best practices . . . 78
12.5 Practice exercises . . . 79
Solutions . . . 79
12.6 Mini test . . . 79
Answer key and explanations . . . 79
**Cheat sheet . . . 80**
13.1 Units and conversions . . . 80
13.2 Networks . . . 80
13.3 Gas . . . 80
13.4 Solidity quick reference . . . 80
13.5 Hashes and signatures . . . 81
13.6 Token standards . . . 81
13.7 OpenZeppelin imports . . . 82
13.8 Clients . . . 82
13.9 Hardhat and Foundry . . . 82
13.10 The DAO runbook . . . 83
**Final test . . . 83**
14.1 Final test answers . . . 84
**Sources . . . 85**

# How to use this volume

Prerequisites: you should be comfortable with JavaScript (Vol. 03 of this library) and with a terminal. No cryptography or economics background is assumed. You do not need to own any cryptocurrency to follow the examples.

Estimated study time: about 25 hours, including the exercises and the final test.

What this volume teaches: what a blockchain is, how accounts and transactions work on Ethereum, how to write and test smart contracts in Solidity, how to build a small dApp that talks to them, how to read chain data, and how to avoid the classic security mistakes.

How the chapters work: every chapter opens with a summary, explains the concepts, shows numbered examples with the output they really produced, lists the common mistakes with best practices, and closes with practice exercises, their solutions, and a mini test with an answer key.

A note on verification. This volume was produced from the free official Ethereum documentation, because the originally planned course site was unreachable. Every Solidity, JavaScript and bash example in these chapters was executed with solc 0.8.28 through Hardhat 3.19.0, running against a local `npx hardhat node`, using ethers 6.17.0 and viem 2.57.4, on Node.js 22.21.0. Blocks marked with a `// NOTE` comment could not be executed in that environment - they need a browser with a wallet extension, an IPFS daemon, a live testnet, or a Foundry installation that only exists on Linux and macOS - and their output is described in words instead of invented.

Tip. Work through chapters 1 to 6 before writing any contract of your own. The toolchain chapters exist so that every later example is something you can run and change yourself.

How to run the code

**shell**

```shell
mkdir web3-study && cd web3-study
npm init -y
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox-viem
npm install ethers viem
npx hardhat compile        # compile the contracts
npx hardhat test           # run the test suite
npx hardhat node           # start a local chain on http://127.0.0.1:8545
npx hardhat run scripts/deploy.ts            # deploy to an in-process chain
npx hardhat run scripts/deploy.ts --network localhost   # deploy to hardhat node
```

## Chapter map

| # | Chapter | You will learn |
|---|---|---|
| 1 | What Web3 is | decentralization, blocks, hash chains, consensus, gas |

| # | Chapter | You will learn |
|---|---|---|
| 2 | Cryptography and accounts | keys, addresses, wallets, transactions, fees |
| 3 | The Ethereum ecosystem | the EVM, clients, networks, testnets, JSON-RPC |
| 4 | Solidity basics | types, functions, visibility, events, errors |
| 5 | Data structures and inheritance | mappings, structs, arrays, modifiers |
| 6 | The toolchain | Hardhat, compiling, local chains, first deployment |
| 7 | Writing tokens | ERC-20, ERC-721, ERC-1155 with OpenZeppelin |
| 8 | Building a dApp frontend | wallets, ethers and viem, the transaction flow |
| 9 | Reading blockchain data | events, logs, The Graph, IPFS |
| 10 | Security, patterns, upgradeability | reentrancy, checks-effects-interactions, proxies |
| 11 | DeFi and DAO building blocks | swaps, staking, voting, governance |
| 12 | Testing, deploying, beyond Ethereum | tests, verification, mainnet, other chains |

# Chapter 1 - What Web3 is

## 1.1 Summary

- A blockchain is a shared database whose history is grouped into blocks that are locked together with cryptographic hashes.
- Each block points at the previous block, so rewriting an old block would force you to rewrite every block after it.
- Decentralization means no single company owns the database; many independent computers keep and check the same copy.
- Consensus is the rule set that lets strangers who distrust each other agree on the next block.
- Ethereum has used proof of stake since September 2022: validators lock up 32 ETH and are rewarded for proposing and attesting blocks.
- Slots arrive about every 12 seconds, and blocks become effectively final after two epochs, which is a little over 12 minutes.
- Gas is the unit of computation: every operation costs a small amount, and the sender pays for the total.
- Web3 is the stack built on top of this: smart contracts, tokens, wallets, decentralized finance and organizations, and the apps that use them.

## 1.2 Concepts

### A chain of blocks

A block bundles a list of transactions plus metadata such as its number, the time it was produced, and the hash of the block before it. The block hash covers all of that content, so any change to the data changes the hash and breaks the link.

**output**

```
+---------------------+        +---------------------+        +---------------------+
| block 100           |        | block 101           |        | block 102           |
| parent: hash of 100 |------->| parent: hash of 101 |------->| parent: hash of 102 |
| 214 transactions    |        | 187 transactions    |        | 203 transactions    |
+---------------------+        +---------------------+        +---------------------+
         hash covers everything above the link line, including the parent hash
```

Anyone holding a copy of the chain can verify this structure from the genesis block forward, which is why people say a blockchain is a trust-minimized database: you verify the data yourself instead of believing an operator.

## Centralized versus decentralized bookkeeping

| Property | Centralized database | Public blockchain |
|---|---|---|
| Who writes | the operator | anyone with a valid transaction and gas |
| Who keeps copies | the operator and its backups | thousands of independent nodes |
| Who can audit | those given access | anyone, at any time |
| Failure of one actor | service can stop or rewrite | the network continues |
| Cost per write | near zero | priced in gas |

A blockchain trades efficiency for censorship resistance and auditability. That trade is the entire reason the technology exists.

## Consensus without a boss

Two families of rules dominate:

- Proof of work. Specialized machines compete to solve a pointless puzzle; the winner proposes the next block. Bitcoin still uses it.
- Proof of stake. Validators deposit capital and are chosen by a pseudo-random procedure to propose blocks; other validators attest that the block is valid. Dishonest behavior can be penalized by destroying part of the deposit.

Ethereum chose proof of stake in a 2022 upgrade often called the Merge, because the original proof-of-work chain was joined with the new proof-of-stake beacon chain. The practical numbers you should remember: a deposit of 32 ETH per validator, a block every 12 seconds, and near-finality after roughly two epochs of 32 slots each.

## Gas pays for shared resources

Computing is not free, and an open network has finite capacity. Ethereum prices every operation in gas and charges the transaction sender. A simple value transfer costs 21,000 gas because it is a fixed, trivial operation. Anything that touches contract storage costs more, and anything that loops costs proportionally.

## 1.3 Examples

Example 1: building a hash chain with Node's own crypto module. This is plain JavaScript and it runs anywhere Node runs.

**JavaScript**

```javascript
import { createHash } from "node:crypto";

function sha256(text) {
  return createHash("sha256").update(text).digest("hex");
}

let previous = "0".repeat(64);
for (let i = 1; i <= 3; i++) {
  const block = previous + ":" + i + ":transfer 2 ETH from A to B";
  const hash = sha256(block);
  console.log(`block ${i}`);
  console.log("  data to hash :", block);
  console.log("  block hash   :", hash);
  console.log("  hash[:16]    :", hash.slice(0, 16));
  previous = hash;
}
```

**output**

```
block 1
  data to hash : 0000000000000000000000000000000000000000000000000000000000000000:1:transfer 2 ETH from
  block hash   : 124875fe7c6cd1de686152b942eb34245950f9c4b9774b0203937287da844466
  hash[:16]    : 124875fe7c6cd1de
block 2
  data to hash : 124875fe7c6cd1de686152b942eb34245950f9c4b9774b0203937287da844466:2:transfer 2 ETH from
  block hash   : c131ab258ce792c9f81d61915133e67f3daebf89499e31f0e3890abe1b6d587a
  hash[:16]    : c131ab258ce792c9
block 3
  data to hash : c131ab258ce792c9f81d61915133e67f3daebf89499e31f0e3890abe1b6d587a:3:transfer 2 ETH from
  block hash   : 2dccefdb2bb52885c1b0bae31ca8b842b45834fc74afff392f34262ead94b46b
  hash[:16]    : 2dccefdb2bb52885
```

Notice that block 3's hash only looks like that because it includes block 2's hash, which only looks like that because it includes block 1's. Changing one character in block 1 invalidates every later hash.

Example 2: what a block looks like on the wire. A local chain was queried right after it started. A freshly started block has height `0x0`, a base fee of one gwei, and no transactions.

**JSON**

```json
{
  "number": "0x0",
  "hash": "0x...(differs on every chain)",
  "parentHash": "0x0000000000000000000000000000000000000000000000000000000000000000",
  "timestamp": "0x...(seconds since 1970, differs per run)",
  "gasLimit": "0x3938700",
  "gasUsed": "0x0",
  "baseFeePerGas": "0x3b9aca00",
  "transactions": []
}
```

Numbers and hashes arrive as hex quantities: `0x3938700` is 60,000,000 gas of limit, and `0x3b9aca00` is one gwei, that is 10^9 wei. A genesis block is the only block whose parent hash is all zeroes. Values marked with a comment differ per chain or per run and are deliberately not repeated.

Example 3: how a fee adds up. A plain transfer pays a fixed 21,000 gas; the price per gas is the base fee plus a tip to the validator.

**JavaScript**

```
const gasLimit = 21000n;
const baseFee = 12_000_000_000n;     // 12 gwei in wei
const tip = 2_000_000_000n;          // 2 gwei in wei
const feeWei = gasLimit * (baseFee + tip);
const feeEth = Number(feeWei) / 1e18;
console.log("fee in wei:", feeWei.toString());
console.log("fee in ETH:", feeEth);
```

**output**

```
fee in wei: 294000000000000
fee in ETH: 0.000294
```

If the base fee doubles and the sender does nothing, the same transfer costs 0.000588 ETH. Wallets estimate the current values for you; this arithmetic is what they are estimating.

Example 4: asking a node a question over JSON-RPC with curl. The answer is hex.

**shell**

```
curl -s -X POST http://127.0.0.1:8545 \
  -H "Content-Type: application/json" \
  --data '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'
```

**output**

```
{"jsonrpc":"2.0","id":1,"result":"0x7a69"}
```

`0x7a69` is 31337, the chain ID that `npx hardhat node` and Foundry's `anvil` use for a local chain. Chapter 3 covers the public networks and their IDs.

## 1.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Thinking a blockchain is a normal database you can edit | corrections must be new entries, never rewrites | model history as append-only |
| Sending real money while learning | mistakes are irreversible | use a local or testnet chain until the code works |
| Ignoring gas when writing loops | unbounded loops can price a call out of reach | cap iterations, paginate work |
| Assuming every chain has the same rules | IDs, finality and fees differ per network | read the chain ID and configure each network |

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Trusting a block explorer screenshot as proof | explorers render chain data | verify against the chain itself |

- Draw the system before coding it: who sends transactions, who pays, what is stored on-chain and what stays off-chain.
- Remember that "decentralized" describes the network, not automatically your application: a dApp can still have a centralized website in front of it.

## 1.5 Practice exercises

1. In example 1, change one character in the first block's data and describe what happens to blocks 2 and 3.
2. A block has `gasLimit` 0x3938700. Convert it to decimal and to million gas.
3. Why does a plain transfer cost exactly 21,000 gas?
4. Compute the fee for 60,000 gas at 25 gwei, showing wei and ETH.
5. Explain in two sentences why rewriting an old block is impractical.

### Solutions

1. Every later hash changes, because each block's data includes the previous hash. The chain diverges from any honest copy and is rejected.
2. 0x3938700 is 60,000,000 in decimal, which is 60 million gas.
3. Because a value transfer has no computations attached to it: the fee is a flat base cost fixed by the protocol, covering signature checks and balance updates.
4. 60,000 x 25 x 10^9 = 1.5 x 10^15 wei, which is 0.0015 ETH.
5. Each block hash includes the previous block's hash, so to change block 100 you must recompute block 101, 102 and so on. An attacker would have to out-produce the rest of the network for every block from there to now.

## 1.6 Mini test

1. Multiple choice: what locks a blockchain's history in place? - a) a central notary - b) each block hashing the previous block's contents - c) a daily backup - d) a signed receipt from the operator
2. True or false: since 2022 Ethereum finalizes blocks with proof of stake.
3. Multiple choice: how much ETH must a validator deposit on Ethereum? - a) 1 ETH - b) 32 ETH - c) 128 ETH - d) no deposit is needed
4. Short answer: what is a slot target time on Ethereum, in seconds?
5. Multiple choice: what does gas measure? - a) the price of ETH - b) the amount of computation and storage an operation uses - c) the number of nodes online - d) the block reward
6. True or false: on a public blockchain only the operator can audit the full history.

7. Short code question: in plain JavaScript, print the hex chain ID 31337.

### Answer key and explanations

1. b - the parent hash chain makes any edit visible in every later hash.
2. True - the Merge in September 2022 moved Ethereum to proof of stake.
3. b - 32 ETH per validator.
4. 12 seconds per slot, on average.
5. b - gas meters execution resources; the fee is gas times a price.
6. False - anyone can download and verify the whole chain.
7. `console.log((31337).toString(16));` prints `7a69`.

# Chapter 2 - Cryptography and accounts

## 2.1 Summary

- An account is controlled by a private key, and the private key is nothing more than a very large random number.
- Ethereum uses the secp256k1 curve: the public key is derived from the private key, and the address is the last 20 bytes of the hash of the public key.
- Externally owned accounts are human- or program-controlled; contract accounts hold code and can only act when called.
- A transaction is a signed instruction that says who to charge, what to do, and how much computation the sender is willing to pay for.
- Nonces order transactions from the same account and stop replays.
- EIP-1559 splits the fee into a burned base fee and a tip for the validator.
- Hierarchical deterministic wallets expand one seed phrase into many accounts using the path m/44'/60'/0'/0/index.
- A seed phrase regenerates every account, so it is the only secret that must never be stored, shared or committed.

## 2.2 Concepts

### From key to address

```
ext random 256 bits | v private key --(secp256k1 multiply)--> public key (64 bytes) | v
keccak256(public key) | take the last 20 bytes | v address (0x + 40 hex chars)
```

A Ethereum address is 20 bytes, written as 42 hex characters. Mixed case in a displayed address is not decoration: EIP-55 defines a checksum where the letters are upper-cased according to the address hash, so a typo can be detected by software. A checksummed address shows that the tool verified the pattern, not that the address exists on-chain.

### Two kinds of account

| Property | Externally owned account | Contract account |
|---|---|---|
| Controlled by | a private key | its code |
| Starts transactions | yes | no, only replies to calls |
| Holds a balance | yes | yes |
| Has code | no | yes |

You can tell them apart by asking a node for their code: an EOA answers with □x, a contract answers with bytecode. Chapter 3 shows the call.

### Anatomy of a transaction

| Field | Meaning |
|---|---|
| once | the sender's transaction count, used for ordering and replay protection |
| o | recipient address, empty for a contract creation |
| alue | amount of wei transferred |
| data | calldata for the call, or creation bytecode when o is empty |
| maxFeePerGas | ceiling the sender pays per unit of gas |
| maxPriorityFeePerGas | tip included to speed inclusion |
| gasLimit | maximum gas the sender authorizes; unused gas is refunded |
| chainId | network identifier, stops replaying a signature on another chain |
| , | |
| , s | the signature parts, recoverable into the sender address |

Transactions are broadcast to the network, held in a public mempool, then included in a block. Only inclusion makes them real: until then a transaction can fail, be replaced, or never land.

### Hierarchical deterministic wallets

A wallet seed is 12 to 24 words generated from randomness. From the seed, the BIP-32 and BIP-44 standards derive a tree of keys; Ethereum accounts use the path m/44'/60'/0'/0/index. The same words in any compatible wallet produce the same accounts, which is why losing the device does not mean losing funds, and why anyone with the words controls the funds.

## 2.3 Examples

Example 1: deriving the first two accounts from a standard test seed. This needs ethers, installed with pm install ethers, and the phrase shown is the one a Fresh local chain such as px hardhat node ships with.

`js import { ethers } from "ethers";

const mnemonic = ethers.Mnemonic.fromPhrase( "test test test test test test test test test test test junk", ); const a0 = ethers.HDNodeWallet.fromMnemonic(mnemonic, "m/44'/60'/0'/0/0"); const a1 = ethers.HDNodeWallet.fromMnemonic(mnemonic, "m/44'/60'/0'/0/1"); console.log("account 0:", a0.address); console.log("account 1:", a1.address); `

```
ext account 0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 account 1:
0x70997970C51812dc3A010C7d01b50e0d17dc79C8
```

These are exactly the addresses px hardhat node prints at startup, each prefunded with 10,000 test ETH. Test funds only ever exist on test networks.

Example 2: checksums and the Transfer event signature. Keccak256 is Ethereum's hash function; it is not the same as the SHA-3 family you find in Node's crypto module, which is why this snippet uses ethers rather than node:crypto.

`js import { ethers } from "ethers";

console.log("checksummed:", ethers.getAddress("0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266")); console.log("transfer topic:", ethers.id("Transfer(address,address,uint256)")); console.log("keccak256 of the word hello:", ethers.keccak256(ethers.toUtf8Bytes("hello"))); `

```
ext checksummed: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 transfer topic:
0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef keccak256 of the word
hello: 0x1c8aff950685c2ed4bc3174f3472287b56d9517b9c948127319a09a7a36deac8
```

The middle line is the keccak256 hash of the event signature, and it becomes opics[0] of every ERC-20 Transfer log. Chapter 9 uses it to filter logs.

Example 3: telling an EOA from a contract by reading its code. This talks to a local node started with px hardhat node.

`js import { ethers } from "ethers";

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545"); const eoa = "0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266";

const code = await provider.getCode(eoa); console.log("code of the EOA:", JSON.stringify(code), "length:", code.length); `

```
ext code of the EOA: "0x" length: 2
```

□x means no code, so this is an externally owned account. A deployed contract would return several hundred bytes starting with 60806040, the standard dispatcher of compiled Solidity.

Example 4: a real cost check. After deploying a tiny counter contract, the receipt reports the gas used and the price actually paid.

`js import { ethers } from "ethers";

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545"); const wallet = new ethers.Wallet( "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80", provider, ); // counter is an ethers Contract already deployed against this chain const receipt = await (await counter.inc()).wait(); console.log("gas used:", receipt.gasUsed.toString());

console.log("price paid (gwei):", ethers.formatUnits(receipt.gasPrice, "gwei"));
console.log("cost (ETH):", ethers.formatEther(receipt.gasUsed * receipt.gasPrice)); `

```
ext gas used: 45130 price paid (gwei): 1.766151339 cost (ETH): 0.00007970640992907
```

A storage write dominates that total: reading is cheap, writing state is the expensive part, and that is the main reason contract design matters.

## 2.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Committing a private key to a repository | anyone reading it owns the account | environment variables, never code |
| Reusing one key everywhere | a single leak empties everything | one role per key, hardware keys for value |
| Replaying a transaction by mistake | double spend, or a call executed twice | nonces and chain IDs exist to prevent this |
| Setting a very low fee tip on a busy network | the transaction can sit for a long time | use the wallet's estimate, then wait |
| Sending ETH before checking the address format | a malformed address silently loses funds | checksum-check every address in code |
| Storing the seed phrase in a screenshot or note | full compromise of the wallet tree | paper or a hardware device, offline |

- Treat a private key like a live weapon: no copies, no commits, no messages.
- Test-only accounts are fine to hard-code, and only on networks with no value.

## 2.5 Practice exercises

1. Which two steps produce an address from a private key?
2. Why is getAddress with a lowercase input still useful?
3. What does a

once of 7 from an account tell you?

4. A transaction sets gasLimit 100,000 and uses 45,130. What happens to the difference?
5. In two sentences, why is a seed phrase dangerous to store next to a project?

### Solutions

1. The public key is derived with the secp256k1 multiplication, then the keccak256 hash of that public key is computed and the last 20 bytes become the address.
2. Because it returns the EIP-55 checksummed form, which tools can use to detect transcription errors, even though the on-chain bytes are identical.
3. That seven transactions from that account have already been included, so this is the eighth.

4. Unused gas is refunded; the sender is only charged for gas actually consumed.
5. The phrase deterministically regenerates every account in the tree, so it is a complete copy of all funds and permissions, usable by anyone from anywhere.

## 2.6 Mini test

1. Multiple choice: which curve does Ethereum use for signatures? - a) ed25519 - b) secp256k1 - c) P-256 - d) curve25519
2. True or false: a contract account can start a transaction by itself.
3. Multiple choice: an address is how many bytes? - a) 20 - b) 32 - c) 16 - d) 64
4. Short answer: which field prevents replaying a transaction on another network?
5. True or false: the same seed phrase gives the same accounts in any compatible wallet.
6. Multiple choice: what does provider.getCode(address) returning □x mean? - a) the node failed - b) the address has no code and is an externally owned account - c) the address is a token - d) the chain is syncing
7. Short code question: with ethers, convert 2.5 ETH to wei in one line.

### Answer key and explanations

1. b - secp256k1, the same curve Bitcoin uses.
2. False - contracts only execute when a transaction calls them.
3. a - 20 bytes, displayed as 42 characters including the 0x prefix.
4. chainId, because the signature commits to the network it came from.
5. True - deterministic derivation is the whole point of HD wallets.
6. b - empty code means no contract is deployed there.
7. ethers.parseEther("2.5"); gives 2500000000000000000n.

# Chapter 3 - The Ethereum ecosystem

## 3.1 Summary

- The Ethereum Virtual Machine is the sandboxed computer that runs contract code on every node at the same time.
- EVM code is bytecode, a stream of opcodes that operate on a stack; storage is the durable, expensive part of the machine.
- Execution clients hold the state and process transactions, while consensus clients agree on the order of blocks.
- Networks are identified by a chain ID, and mixing them up is a classic production incident.
- Sepolia and Holesky are the permissionless test networks used for practice.
- Faucets hand out test ETH so you can pay gas without using real money.

- Nodes speak JSON-RPC over HTTP or WebSocket, and every wallet library is a thin client on top of those methods.
- Block explorers read a node and present the results in a human-readable way.

## 3.2 Concepts

### What the EVM is

The EVM is a stack machine with 256-bit words. Each opcode does one small thing - add two numbers, store a word, read a word, call another contract - and each opcode has a fixed gas cost. Because every node executes exactly the same bytecode in the same order, they all reach the same state, which is what makes the shared database trustworthy.

```
ext persistent storage (world state) address -> account (balance, nonce, codeHash,
storageRoot) | v contract bytecode --opcodes--> memory (temporary, per call) | v stack (per
call, 256-bit words)
```

Storage is key-value and it lives forever unless the code changes it. Memory and the stack are cleared between calls. This difference explains why writing to storage is the most expensive operation a contract performs.

### Clients and nodes

- Execution client - implements the EVM, the transaction pool and the state database. Examples: Geth, Nethermind, Erigon, Besu.
- Consensus client - implements the proof-of-stake rules: attestations, epochs, finality. Examples: Prysm, Lighthouse, Teku, Nimbus.
- They run as a pair, connected locally, and together they are a node.

You do not need to run a node to learn. A local chain such as px hardhat node implements both sides in one process, and an RPC provider runs real nodes for you and rents you access.

### The networks you will meet

| Network | Chain ID | Purpose |
|---|---|---|
| Ethereum mainnet | 1 | real ETH, real money |
| Sepolia | 11155111 | permissionless proof-of-stake testnet |
| Holesky | 17000 | larger testnet for validator practice |
| Local Hardhat node | 31337 | your machine, instant blocks, disposable funds |

Chain IDs are 32-bit numbers and unrelated to each other. A signature signed for chain 1 is invalid on 11155111, which is exactly the protection you want when you practice next to production code.

### The RPC surface

Everything a client library does is expressed through methods such as eth_chainId, eth_blockNumber, eth_getBalance, eth_call, eth_estimateGas, eth_sendRawTransaction and eth_getLogs. Quantities are hex strings, addresses are 20-byte hex, and blocks can be

requested by number, by tag such as latest, or by hash.

## 3.3 Examples

Example 1: asking a fresh local node for its chain ID with curl.

```
ash curl -s -X POST http://127.0.0.1:8545 \ -H "Content-Type: application/json" \ --data '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'
```

```
ext {"jsonrpc":"2.0","id":1,"result":"0x7a69"}
```

Example 2: the same node, three questions, from plain Node. etch is built in, so no library is needed.

`js async function rpc(method, params = []) { const res = await fetch("http://127.0.0.1:8545", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ jsonrpc: "2.0", method, params, id: 1 }), }); return res.json(); }

console.log("chainId:", (await rpc("eth_chainId")).result); console.log("blockNumber:", (await rpc("eth_blockNumber")).result); console.log("balance:", (await rpc("eth_getBalance", ["0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266", "latest"])).result); `

```
ext chainId: 0x7a69 blockNumber: 0x0 balance: 0x21e19e0c9bab2400000
```

The balance is wei: x21e19e0c9bab2400000 is 10,000 ETH, the amount each of the 20 development accounts receives. A fresh chain starts at block x0.

Example 3: block metadata tells you how the network is behaving.

```
js const block = await rpc("eth_getBlockByNumber", ["latest", false]); console.log("baseFeePerGas:", block.result.baseFeePerGas); console.log("gasLimit:", block.result.gasLimit);
```

```
ext baseFeePerGas: 0x3b9aca00 gasLimit: 0x3938700
```

A base fee of one gwei on an idle local chain, with a 60 million gas block limit. On mainnet the base fee moves every block with demand.

## 3.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Pointing a deploy script at mainnet by default | real funds move | default to a local network, require an explicit flag |
| Assuming block numbers match between networks | state differs per chain | treat each chain as a separate universe |
| Polling eth_blockNumber in a tight loop | rate limits, wasted calls | subscribe to new heads when the transport allows |
| Treating eth_call results as final | calls do not change state | use eth_call to preview, transactions to act |
| Forgetting test ETH expires or drains | stalled practice | top up from a faucet as needed |

- Read the chain ID at startup and log it; it is the cheapest sanity check against a bad configuration.
- Keep one small RPC helper, like example 2, in every project: it becomes the debugging tool when a library hides an error.

## 3.5 Practice exercises

1. Which two client types must a node run since the Merge?
2. Convert 0x3938700 to decimal and state what it caps.
3. Why does a signature for chain ID 1 fail on Sepolia?
4. What does a base fee of 0x3b9aca00 equal in wei and gwei?
5. Name two things an execution client does that a consensus client does not.

### Solutions

1. An execution client and a consensus client.
2. 60,000,000, the maximum gas a block may consume.
3. The chain ID is part of what the signature commits to, so the same signature is not valid on a different network.
4. 10^9 wei, which is one gwei.
5. Execute transactions and maintain the state storage and the transaction pool; the consensus client handles attestations and finality.

## 3.6 Mini test

1. Multiple choice: which of these is an execution client? - a) Prysm - b) Geth - c) Lighthouse - d) Teku
2. True or false: the EVM maintains two kinds of memory: storage and memory.
3. Multiple choice: what is the chain ID of Sepolia? - a) 1 - b) 5 - c) 11155111 - d) 31337
4. Short answer: which RPC method returns the node's chain ID?
5. Multiple choice: what does 0x21e19e0c9bab2400000 wei equal? - a) 1 ETH - b) 10,000 ETH - c) 1 gwei - d) 10 ETH
6. True or false: eth_call executes a transaction and changes state.
7. Short answer: what is a faucet for?

### Answer key and explanations

1. b - Geth is an execution client; the others are consensus clients.
2. True - durable storage and per-call memory, plus the stack.
3. c - Sepolia uses 11155111; 31337 is a local Hardhat node.
4. eth_chainId.
5. b - it is the development balance of 10,000 ETH in wei.

6. False - a call is read-only and simulated locally.
7. It gives away free test-network ETH so you can pay practice gas fees.

# Chapter 4 - Solidity basics

## 4.1 Summary

- Solidity is the most used language for Ethereum contracts: statically typed, compiled to EVM bytecode, and object-oriented.
- A file starts with a license identifier and a pragma that pins the compiler range.
- Value types include booleans, integers of fixed width, addresses, bytes of fixed width, and enums; reference types are arrays, structs, mappings and strings.
- Functions declare their visibility, their state mutability, and their parameters; the compiler rejects mismatches.
- Events are cheap logs that let off-chain code follow what the contract did.

equire reverts with a reason string, while custom errors are cheaper and typed.

- Since version 0.8, arithmetic overflow is checked by default, so old SafeMath-style libraries are no longer needed.
- The compiler also produces the ABI, the description every client library uses to encode calls and decode results.

## 4.2 Concepts

### The file frame

`solidity // SPDX-License-Identifier: MIT pragma solidity ^0.8.24;

contract Counter { uint256 public x;

event Incremented(address indexed by, uint256 newValue);

error Overflow();

constructor(uint256 start) { x = start; } } `

^0.8.24 means this code compiles with any 0.8.x compiler from 0.8.24 upward, but not with 0.9. Pinning a range on purpose protects you from a new compiler changing behaviour under a deployed contract. A contract with no constructor still has one implicitly.

### Types worth memorizing

| Type | Example | Notes |
|---|---|---|
| ool | rue | costs one byte in storage |
| uint256 / int256 | 42, -7 | fixed width, overflow checks on 0.8+ |
| ddress | xabc... | 20 bytes, ddress(this) is the contract |

| Type | Example | Notes |
|---|---|---|
| ddress payable | payable(user) | may receive ETH, and can .transfer |
| ytes32 | ytes32("AB") | fixed bytes, common for hashes |
| string | "hello" | dynamic, expensive to keep in storage |
| enum | enum State { Idle, Run } | small integers with names |
| arrays | uint256[], uint8[3] | dynamic or fixed length |
| mapping | mapping(address => uint256) | hash table, cannot be iterated |
| struct | struct P { uint256 a; } | a group of named fields |

### Functions

Four visibility levels decide who may call:

| Visibility | Reachable from |
|---|---|
| external | other contracts and transactions only |
| public | the contract itself and the outside |
| internal | this contract and contracts that inherit it |
| private | this contract only, not even derived ones |

Three mutability levels change what is allowed:

| Mutability | Meaning |
|---|---|
| pure | reads neither state nor the environment |
| iew | reads state or environment, writes nothing |
| payable | may receive ETH as part of the call |

Inside a function, msg.sender is the immediate caller, msg.value is the sent ETH, lock.timestamp is the block time, and x.origin is the transaction origin - which you should distrust, as chapter 10 explains.

### Events and errors

An event copies its data into a log, where clients can read it cheaply. Only indexed parameters can be filtered efficiently, and at most three can be indexed. An error either is a string through equire or a typed custom error declared next to the contract.

## 4.3 Examples

Example 1: the counter contract this chapter teaches from.

`solidity // SPDX-License-Identifier: MIT pragma solidity ^0.8.24;

contract Counter { uint256 public x;

event Incremented(address indexed by, uint256 newValue);

function inc() public { x++; emit Incremented(msg.sender, x); } } `

Example 2: compile it and read the result. The line below is what Hardhat printed with solc 0.8.28 for this single file.

```
ash npx hardhat compile

ext Compiled 1 Solidity file with solc 0.8.28 (evm target: cancun)
```

The build folder now holds the artifact, including the ABI that clients and the test suite use to talk to the contract.

Example 3: deploy and call it from a script against a local node.

`js import { readFileSync } from "node:fs"; import { ethers } from "ethers";

const artifact = JSON.parse( readFileSync("./artifacts/contracts/Counter.sol/Counter.json", "utf8"), );

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545"); const wallet = new ethers.Wallet( "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80", provider, );

const factory = new ethers.ContractFactory(artifact.abi, artifact.bytecode, wallet); const counter = await factory.deploy(); await counter.waitForDeployment();

console.log("deployed at:", await counter.getAddress()); console.log("x before:", (await counter.x()).toString());

const tx = await counter.inc(); const receipt = await tx.wait(); console.log("gas used:", receipt.gasUsed.toString()); console.log("x after:", (await counter.x()).toString()); `

```
ext deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3 x before: 0 gas used: 45130 x
after: 1
```

The address is the standard first deployment of the default development account on a fresh chain; any earlier transaction from that account moves it.

Example 4: a function that refuses, twice. equire carries a human reason, the custom error carries a code that clients can branch on.

`solidity // SPDX-License-Identifier: MIT pragma solidity ^0.8.24;

error NotOwner(address caller);

contract Guarded { address public owner;

constructor() { owner = msg.sender; }

function withdrawAll() external { if (msg.sender != owner) revert NotOwner(msg.sender); }

function ping() external pure { require(true, "ping received"); } } `

Example 5: integer wrapping is history. On solc 0.8 this arithmetic reverts instead of wrapping, which removes a whole bug class by construction.

`solidity // SPDX-License-Identifier: MIT pragma solidity ^0.8.24;

contract Checked { function downUnchecked(uint256 n) external pure returns (uint256) { unchecked { return n - 1; // author asked for raw speed here } }

function downSafe(uint256 n) external pure returns (uint256) { return n - 1; // reverts when n is 0 } } `

unchecked exists for deliberate gas savings, and it moves the responsibility for the arithmetic onto you.

## 4.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Forgetting payable when a function takes ETH | the transfer reverts | mark receiving functions payable |
| public everywhere | more expensive and more exposed | external for functions only called from outside |
| Long strings in storage | high deployment gas | store hashes, keep text off-chain |
| Iterating a mapping | impossible; there is no key list | keep a separate array of keys |
| Arrays in calldata copied to storage | silent, expensive copies | use calldata for read-only parameters |
| x.origin for authorization | phishing can ride it | always check msg.sender |

- Name events in the past tense and index the columns you will filter on, not everything that fits.
- Return values from iew functions are free for you locally but still cost gas when called by another contract.

## 4.5 Practice exercises

1. Write a uint256 variable incremented by a public function, then an event.
2. Which visibility allows a function to be called only from contracts that inherit this one?
3. What does unchecked change about  - b in solc 0.8?
4. Why is mapping(address => uint) not enough to list all holders?
5. Which modifier lets a function accept native ETH?

### Solutions

1. uint256 public total; plus unction add() external { total++; emit Added(total); } with event Added(uint256 newTotal); declared.
2. internal.
3. It removes the automatic overflow or underflow check, so the arithmetic wraps silently as it did before 0.8.

4. A mapping is a hash table without an ordered key list; you must track keys in an array if you need to enumerate them.
5. payable.

## 4.6 Mini test

1. Multiple choice: which line starts the license metadata in a Solidity file? - a) `pragma solidity ^0.8.24;` - b) `// SPDX-License-Identifier: MIT` - c) `import "ERC20.sol";` - d) `contract MIT {}`
2. True or false: since 0.8, `uint256` subtraction underflow reverts by default.
3. Multiple choice: which visibility blocks other contracts from calling a function at all? - a) `public` - b) `external` - c) `private` - d) `internal`
4. Short answer: how many event parameters may be `indexed`?
5. True or false: `pure` functions may read a state variable.
6. Multiple choice: which comparison replaces `tx.origin == owner` safely? - a) `msg.sender == owner` - b) `block.coinbase == owner` - c) `tx.gasprice == 0` - d) `address(this) == owner`
7. Short code question: declare a public state variable named `balance` of type `uint256`.

### Answer key and explanations

1. b - the SPDX comment; the pragma is the next line and pins the compiler.
2. True - checked arithmetic is on by default.
3. c - `private` is visible only inside the contract itself.
4. Three, because the log structure allows three indexed topics plus the signature topic.
5. False - `pure` may not read state; reading only is `view`.
6. a - `msg.sender` is the immediate caller; `tx.origin` can be a victim contract.
7. `uint256 public balance;`

# Chapter 5 - Data structures and inheritance

## 5.1 Summary

- Structs group named fields into one logical record, and they live in storage when they are state variables.
- Mappings are hash tables keyed by any value type, with no size, no order, and no way to list the keys.
- Dynamic arrays grow with `push`, fixed arrays have a length known at compile time, and both can live in storage, memory, or calldata.
- Data location decides cost: `storage` is durable, `memory` is per call, and `calldata` is the incoming arguments, read-only.
- Modifiers wrap functions and are the natural place for access checks.

- Inheritance copies state variables and functions into the derived contract, and `virtual` plus `override` control which implementation wins.
- Abstract contracts and interfaces declare shape without full implementation, and an interface cannot have state or implemented functions.
- Naming collisions are resolved explicitly, which is why multiple inheritance in Solidity is said to be linearized.

## 5.2 Concepts

### Structs and mappings together

Most contracts pair a `mapping` as an index with a `struct` as the record:

**Solidity**

```
struct Proposal {
    string title;
    uint256 votes;
    bool open;
}

mapping(uint256 => Proposal) public proposals;
uint256 public proposalCount;
```

The public getter for `proposals` returns the struct's scalar fields but not dynamic ones such as `title`, because a public mapping getter cannot return a string. Provide a view function that returns the whole struct when you need it.

### Data locations

| Location | Lifetime | Cost | Typical use |
|---|---|---|---|
| `storage` | forever, on-chain | most expensive | state variables |
| `memory` | the current call | gas to allocate and copy | temporary values |
| `calldata` | the current call | cheapest, read-only | function parameters from outside |

Passing a struct from `calldata` to `memory` copies it; passing it to `storage` by reference does not. Copying by accident is a common and invisible gas cost.

### Modifiers

**Solidity**

```
modifier onlyOwner() {
    require(msg.sender == owner, "not the owner");
    _;
}

function pause() external onlyOwner {
    paused = true;
}
```

The underscore marks where the wrapped function's body runs. Modifiers run before and after that point, so they are a clean place for checks and for guards. Avoid heavy logic in a

modifier: it executes on every call, and nesting several modifiers makes the execution order hard to read.

### Inheritance

**output**

```
Ownable (public owner, transferOwnership)
   |
   +--> Token (adds balances, inherits owner checks)
             |
             +--> StakedToken (adds staking, overrides _update)
```

A derived contract receives all non-private members of its bases. Functions declared `virtual` can be replaced, and the replacement must say `override`. Calling the parent version is explicit with `super`.

An `abstract` contract has at least one unimplemented function and cannot be deployed alone. An `interface` is an abstract contract with only external function declarations; it cannot declare state variables, constructors, or implemented functions, and it is how the ecosystem describes shared standards.

## 5.3 Examples

Example 1: the ContactBook contract used in this chapter, with a base contract, a modifier, a struct, an array, and an overridden function.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

abstract contract BaseRegistry {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Registry: caller is not the owner");
        _;
    }

    function describe() public pure virtual returns (string memory);
}

contract ContactBook is BaseRegistry {
    struct Contact {
        string name;
        string wallet;
        bool active;
    }

    Contact[] private contacts;

    event ContactAdded(uint256 indexed id, string name);
    event ContactDeactivated(uint256 indexed id);

    function addContact(string calldata name, string calldata wallet)
        external
        onlyOwner
    {
        contacts.push(Contact({ name: name, wallet: wallet, active: true }));
        emit ContactAdded(contacts.length - 1, name);
    }

    function deactivate(uint256 id) external onlyOwner {
        contacts[id].active = false;
        emit ContactDeactivated(id);
    }

    function getContact(uint256 id)
        external
        view
        returns (string memory name, string memory wallet, bool active)
    {
        Contact storage contact = contacts[id];
        return (contact.name, contact.wallet, contact.active);
    }

    function contactCount() external view returns (uint256) {
        return contacts.length;
    }

```

**Solidity (continued)**

```
    function activeCount() external view returns (uint256 count) {
        for (uint256 i = 0; i < contacts.length; i++) {
            if (contacts[i].active) {
                count++;
            }
        }
    }

    function describe() public pure override returns (string memory) {
        return "ContactBook: an owner-managed list of contacts";
    }
}
```

Example 2: driving it from a script against a local node.

**JavaScript**

```ts
import { readFileSync } from "node:fs";
import { ethers } from "ethers";

const artifact = JSON.parse(
  readFileSync("./artifacts/contracts/ContactBook.sol/ContactBook.json", "utf8"),
);

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const ownerWallet = new ethers.Wallet(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
  provider,
);
const stranger = new ethers.Wallet(
  "0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a",
  provider,
);

const book = await new ethers.ContractFactory(
  artifact.abi,
  artifact.bytecode,
  ownerWallet,
).deploy();
await book.waitForDeployment();

await (await book.addContact(
  "Ada",
  "0x70997970c51812dc3a010c7d01b50e0d17dc79c8",
)).wait();
await (await book.addContact(
  "Alan",
  "0x3c44cdddb6a900fa2b585dd299e03d12fa4293bc",
)).wait();
console.log("count:", (await book.contactCount()).toString());
console.log("active:", (await book.activeCount()).toString());

const contact = await book.getContact(0);
console.log("contact 0:", contact[0], contact[1], contact[2]);

await (await book.deactivate(0)).wait();
console.log("active after deactivate:", (await book.activeCount()).toString());
console.log("contact 0 active:", (await book.getContact(0))[2]);
console.log("describe:", await book.describe());

try {
  await book.connect(stranger).addContact("Eve", "0x90f79bf6eb2c4f870365e785982e1f101e93b906");
  console.log("ERROR: a non-owner added a contact");
} catch (e) {
  console.log("non-owner add reverts:", e.shortMessage ?? e.message);
}

const logs = await book.queryFilter("ContactAdded", 0, "latest");
console.log("ContactAdded events:", logs.length, logs.map((l) => l.args[1]));
console.log("ContactDeactivated events:", (await book.queryFilter("ContactDeactivated", 0, "latest")).l
```

**output**

```
count: 2
active: 2
contact 0: Ada 0x70997970c51812dc3a010c7d01b50e0d17dc79c8 true
active after deactivate: 1
contact 0 active: false
describe: ContactBook: an owner-managed list of contacts
non-owner add reverts: execution reverted: "Registry: caller is not the owner"
ContactAdded events: 2 [ 'Ada', 'Alan' ]
ContactDeactivated events: 1
```

The modifier rejects the stranger before any state changes, and the indexed event fields come back decoded as plain strings.

Example 3: storage, memory and calldata in one contract. The copy into memory is visible in the read-only path, and the reference into storage is not.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Locations {
    struct Item {
        uint256 id;
        uint256 qty;
    }

    Item[] private items;

    function add(uint256 id, uint256 qty) external {
        items.push(Item(id, qty));
    }

    function sumMemory(uint256 upto) external view returns (uint256 total) {
        Item[] memory copy = items;
        for (uint256 i = 0; i < upto && i < copy.length; i++) {
            total += copy[i].qty;
        }
    }

    function sumStorage(uint256 upto) external view returns (uint256 total) {
        for (uint256 i = 0; i < upto && i < items.length; i++) {
            total += items[i].qty;
        }
    }
}
```

`sumMemory` allocates and copies the whole array first, which costs more gas for the same result. `sumStorage` walks the array by reference. Both walk the whole array, so both have a gas cost that grows with the length: any loop over an array that users can append to needs a cap or a different structure.

Example 4: two bases with the same function name, resolved explicitly.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract A {
    function greet() public pure virtual returns (string memory) {
        return "A";
    }
}

contract B is A {
    function greet() public pure virtual override returns (string memory) {
        return "B";
    }
}

contract C is A, B {
    function greet() public pure override(A, B) returns (string memory) {
        return "C, and B says " + super.greet();
    }
}
```

`override(A, B)` lists every base whose function is replaced; the compiler enforces the list. `super.greet()` calls the next implementation in the inheritance order, here `B`.

## 5.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Expecting to list a mapping's keys | mappings have no key list | keep a separate array of keys |
| Copying structs between memory and storage | gas multiplied by record size | pass by reference, copy once |
| Loops over user-extendable arrays | gas grows until the call cannot run | cap the loop or use pagination |
| Logic inside a modifier | hidden costs on every call | keep modifiers to checks |
| Same function name in two bases without `override` | the compiler rejects the build | list all bases explicitly |
| Dynamic types in a public mapping getter | the getter cannot return them | add an explicit view function |

- Model the data first: mappings for lookup, arrays only when order matters.
- Give structs a fixed number of scalar fields where you can; dynamic fields multiply the storage cost.

## 5.5 Practice exercises

1. Why can a public `mapping` getter not return a `string` field?
2. What is the difference between `memory` and `calldata` for a parameter?
3. Which keyword marks a function that derived contracts may replace?

4. Write a modifier `notPaused` that reverts while `paused` is true.
5. Why does `super.greet()` need a linearized inheritance order?

### Solutions

1. Because the auto-generated getter returns only scalar types, and a string is dynamic and would need a second call to return safely.
2. Both live for the duration of the call, but `memory` can be written to and is copied, while `calldata` is the incoming, read-only arguments area and is the cheapest.
3. `virtual` on the base, and `override` on the replacement.
4. `modifier notPaused() { require(!paused, "paused"); _; }`
5. Because when several bases define the same function, the language must know which implementation comes next, so it orders them deterministically and makes you be explicit.

## 5.6 Mini test

1. Multiple choice: which data location is read-only incoming arguments? - a) `storage` - b) `memory` - c) `calldata` - d) `stack`
2. True or false: a `mapping` can be iterated with a `for` loop.
3. Multiple choice: what does `super` refer to? - a) the contract that deployed this one - b) the next base implementation in the inheritance order - c) the owner address - d) the first state variable
4. Short answer: which two keywords mark overridable functions?
5. True or false: an interface may declare a state variable.
6. Multiple choice: which location is the most expensive to write? - a) `memory` - b) `calldata` - c) `storage` - d) the stack
7. Short code question: append an element to a dynamic array named `queue`.

### Answer key and explanations

1. c - `calldata` is the read-only arguments area.
2. False - there is no ordering or key list in a mapping.
3. b - it is the parent implementation, resolved by the fixed inheritance order.
4. `virtual` in the base and `override` in the derived contract.
5. False - interfaces declare shape only, so no state is allowed.
6. c - `storage` persists forever and dominates the fee.
7. `queue.push(element);`

# Chapter 6 - The toolchain

## 6.1 Summary

- Hardhat 3 is a JavaScript workspace: a config file, a compile step, a task runner, and a local network in one dependency.
- A Hardhat 3 project is ESM and typed, so the config is `hardhat.config.ts` and scripts and tests are TypeScript.
- The `@nomicfoundation/hardhat-toolbox-viem` plugin adds the network, tests, verification, and the viem client bindings.
- `npx hardhat compile` writes artifacts with the ABI and bytecode into `artifacts/`, and only recompiles what changed.
- Tests run through Node's own test runner: `node:test`, with the viem client attached to an ephemeral in-process chain.
- `npx hardhat run` executes a script against the in-process network, and `--network localhost` targets a chain started with `npx hardhat node`.
- `npx hardhat node` prints 20 accounts with 10,000 test ETH each and listens on http://127.0.0.1:8545.
- Foundry is the Rust alternative, with its own commands, and it is not installed in the environment used to verify this volume.

## 6.2 Concepts

### The project shape

**output**

```
web3-study/
  hardhat.config.ts     project configuration
  contracts/            Solidity sources, one contract per concern
  test/                 tests, run by `npx hardhat test`
  scripts/              deploy and maintenance scripts
  artifacts/            compiler output: ABI plus bytecode per contract
  cache/                incremental compile state, safe to delete
```

### The config file

This is the complete working configuration used to run every example in this volume. The plugin import line comes from the toolbox package.

**TypeScript**

```ts
import { defineConfig } from "hardhat/config";
import hardhatToolboxViemPlugin from "@nomicfoundation/hardhat-toolbox-viem";

export default defineConfig({
  plugins: [hardhatToolboxViemPlugin],
  solidity: "0.8.28",
  networks: {
    hardhat: { type: "edr-simulated" },
    sepolia: {
      type: "http",
      url: "https://ethereum-sepolia-rpc.publicnode.com",
    },
  },
});
```

Two details are worth remembering. The in-process network is typed `edr-simulated`, which is Hardhat 3's execution environment; and an HTTP network needs a real URL, because a configuration variable or an empty string makes the config fail to load rather than fail at runtime.

### The three commands you use daily

**shell**

```shell
npx hardhat compile   # solc down to artifacts/
npx hardhat test      # run test/ against a fresh in-process chain
npx hardhat run scripts/deploy.ts   # execute a script
```

`npx hardhat node` is the fourth command: it starts a long-running chain you can point a browser wallet or a frontend at, which is harder to do with the in-process network that scripts and tests use.

### Tests with Node's runner

Hardhat 3 delegates tests to `node:test`, so you import `test` from Node and assert with Node's `assert`. The viem client comes from the network connection and can deploy contracts directly.

## 6.3 Examples

Example 1: the counter that every later example reuses.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Counter {
    uint256 public x;

    event Incremented(address indexed by, uint256 newValue);

    function inc() public {
        x++;
        emit Incremented(msg.sender, x);
    }
}
```

Example 2: compile and check the compiler output.

**shell**

```shell
npx hardhat compile
```

**output**

```
Compiled 1 Solidity file with solc 0.8.28 (evm target: cancun)
```

Example 3: a test that deploys, calls, and asserts. The network connection exposes `viem`, the same client used in chapter 8.

**TypeScript**

```
import { test } from "node:test";
import assert from "node:assert/strict";
import hre from "hardhat";

test("increments", async function () {
  const { viem } = await hre.network.connect();
  const counter = await viem.deployContract("Counter");
  await counter.write.inc();
  assert.equal(await counter.read.x(), 1n);
});
```

**output**

```
  âœ” increments
1 passing (1 nodejs)
```

Newer Hardhat 3 releases print a deprecation notice suggesting `hre.network.create()` in place of `hre.network.connect()`; the example still passes, and either call shape is acceptable while the API settles.

Example 4: the deploy script, run twice - once against the throwaway chain and once against a running node.

**TypeScript**

```
import hre from "hardhat";

async function main() {
  const { viem } = await hre.network.connect();
  const counter = await viem.deployContract("Counter");
  console.log("Counter deployed to", counter.address);
  await counter.write.inc();
  console.log("x is now", await counter.read.x());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

**output**

```
Counter deployed to 0x5fbdb2315678afecb367f032d93f642f64180aa3
x is now 1n
```

**shell**

```
npx hardhat node            # in a second terminal, then:
npx hardhat run scripts/deploy.ts --network localhost
```

Addresses depend on the sender's nonce and the deployed code, so a fresh chain shows the canonical first deployment `0x5fbdb...` and later runs show different ones.

Example 5: the accounts a local node hands out. The first block of output looks like this.

**output**

```
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Account #0:  0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

These keys are printed by every local chain because they are public knowledge. Anyone can move the test ETH on any test network they are used on, which is harmless, and it is also why they must never appear in a project that touches a real network.

Example 6: the Foundry alternative. Foundry installs on Linux and macOS through foundryup, so the commands below are taken from the Foundry book and were NOT executed while writing this volume.

**shell**

```
curl -L https://foundry.paradigm.xyz | bash
foundryup

forge init vault && cd vault
forge build
forge test
forge create Counter --rpc-url http://127.0.0.1:8545 \
  --private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
anvil
cast call "$CONTRACT" "x()(uint256)" --rpc-url http://127.0.0.1:8545
cast send "$CONTRACT" "inc()" --rpc-url http://127.0.0.1:8545 \
  --private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

Expected output, word for word: `forge build` reports one or more compiled contracts and writes `out/Counter.sol/Counter.json`; `forge test` runs the tests in `test/`; `anvil` prints the same 20 development accounts as `hardhat node` on port 8545; `cast call` prints a number for `x()` and `cast send` prints the transaction hash. The exact shapes of those lines follow the Foundry book rather than a run in this environment.

## 6.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Compiling against a network by mistake | real deployments from a typo | keep the default local and require an explicit network flag |
| Deleting `artifacts/` while debugging | slower feedback | leave it; it is just a build cache |
| Committing `.env` files with keys | leaked secrets | read from the environment, and add the file to .gitignore |
| Pinning `solc` to `latest` | silent compiler upgrades | a fixed version such as `"0.8.28"` |
| Hardcoding test keys in application code | catastrophic when reused | restrict them to local networks |
| Ignoring a failing deprecation warning | future breakage | fix it when upgrading the toolchain |

- One chain per terminal task: a dev node in one terminal, deploys in another.
- When a task misbehaves, run it with the network name printed out loud; the chain ID tells you instantly where you are.

## 6.5 Practice exercises

1. Which folder holds the ABI after a compile?
2. What does `--network localhost` point at after `npx hardhat node`?
3. Why does the config use a fixed compiler version instead of `latest`?
4. Which command runs the tests in Hardhat 3, and with which runner?
5. What must never be done with the keys a local node prints?

### Solutions

1. `artifacts/`, in a subfolder per source file plus contract name.
2. The node on http://127.0.0.1:8545 that the same project started.
3. Because a fixed version makes builds reproducible; a moving `latest` changes bytecode and can change behaviour under a deployed contract.
4. `npx hardhat test`, which runs Node's built-in `node:test` runner.
5. They must never be used on a network with real funds, and never be pasted into application code or a public repository.

## 6.6 Mini test

1. Multiple choice: which file holds the network configuration? - a) `package.json` - b) `hardhat.config.ts` - c) `.env` - d) `artifacts/config.json`
2. True or false: `npx hardhat test` uses mocha by default in Hardhat 3.
3. Multiple choice: which directory does Hardhat write artifacts to? - a) `build/` - b) `artifacts/` - c) `out/` - d) `dist/`
4. Short answer: which command starts a persistent local chain?
5. True or false: a development account private key is safe to publish.
6. Multiple choice: what does `hardhat.config.ts` default network type look like in Hardhat 3? - a) `type: "edr"` - b) `type: "edr-simulated"` - c) `type: "local"` - d) `type: "dev"`
7. Short code question: in one line, print the viem address of a deployed contract named `counter`.

### Answer key and explanations

1. b - `hardhat.config.ts` is the project configuration.
2. False - Hardhat 3 uses Node's built-in `node:test` runner.
3. b - `artifacts/`, with one JSON artifact per contract.
4. `npx hardhat node`.

5. False - those keys are public and worthless on real networks, and reusing them elsewhere is a security failure.
6. b - the in-process network is typed `edr-simulated`.
7. `console.log(counter.address);` prints something like `0x5fbdb...`.

# Chapter 7 - Writing tokens

## 7.1 Summary

- A token standard fixes the function and event names so that wallets and markets can support every token that follows it.
- ERC-20 is the fungible standard: balances, a fixed supply set by the author, and an allowance system that lets a third party move tokens for you.
- ERC-721 is the non-fungible standard: every token has an identifier, one owner, and optional metadata that lives off-chain.
- ERC-1155 manages many token types, fungible and not, in one contract with batch operations.
- Approval is a powerful delegation, so a dApp that asks for an unlimited allowance deserves suspicion.
- OpenZeppelin ships audited implementations of every standard, with extension points instead of forked code.
- Minting, burning and pausing are privileges, and privileges belong behind ownership or a role check.
- Metadata is usually a JSON file on IPFS or HTTP, and the chain stores only the link.

## 7.2 Concepts

### The three standards side by side

| Property | ERC-20 | ERC-721 | ERC-1155 |
|---|---|---|---|
| Unit | fungible amounts | unique ids | many id families |
| Balance call | `balanceOf(address)` | `balanceOf(address)` | `balanceOf(address, id)` |
| Transfer | `transfer(to, amount)` | `transferFrom` / `safeTransferFrom` | `safeTransferFrom`, `safeBatchTransferFrom` |
| Approval | `approve(spender, amount)` | `approve(to, tokenId)`, `setApprovalForAll` | `setApprovalForAll` |
| Metadata | name, symbol, decimals | `tokenURI(id)` | `uri(id)` with `{id}` |

| Property | ERC-20 | ERC-721 | ERC-1155 |
|---|---|---|---|
| Main events | `Transfer`, `Approval` | `Transfer`, `Approval`, `ApprovalForAll` | `TransferSingle`, `TransferBatch`, `URI` |
| Supply | capped by the author | one per id | per id |

### The allowance pattern

ERC-20 has two ways to move tokens. The owner can call `transfer` directly, or can `approve` a spender, who later calls `transferFrom`. This is how markets and vaults work, and it is also why a single careless approval can drain a wallet.

### Safe transfers and `safeTransferFrom`

ERC-721 has `safeTransferFrom`, which checks that the recipient is either an externally owned account or a contract that says it can receive NFTs. The plain `transferFrom` skips that check, so a token sent to a contract that cannot handle it is stuck there.

### Build on OpenZeppelin

The pattern in this chapter imports the standard from OpenZeppelin and customizes only what changes. Writing a token from scratch in a lesson teaches you the standard; shipping one teaches you the bug classes, which is why audited libraries exist.

## 7.3 Examples

Example 1: an ERC-20 token with a fixed initial supply, owner-gated minting, and burning, built on OpenZeppelin 5.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Burnable} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract StudyToken is ERC20, ERC20Burnable, Ownable {
    constructor(uint256 initialSupply)
        ERC20("Study Token", "STUDY")
        Ownable(msg.sender)
    {
        _mint(msg.sender, initialSupply);
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }
}
```

Example 2: deploying it and exercising the whole API against a local node.

**JavaScript**

```ts
import { readFileSync } from "node:fs";
import { ethers } from "ethers";

const artifact = JSON.parse(
  readFileSync("./artifacts/contracts/StudyToken.sol/StudyToken.json", "utf8"),
);

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const owner = new ethers.Wallet(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
  provider,
);
const alice = new ethers.Wallet(
  "0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d",
  provider,
);

const token = await new ethers.ContractFactory(
  artifact.abi,
  artifact.bytecode,
  owner,
).deploy(ethers.parseUnits("1000", 18));
await token.waitForDeployment();

console.log("name:", await token.name());
console.log("symbol:", await token.symbol());
console.log("decimals:", (await token.decimals()).toString());
console.log("totalSupply:", ethers.formatUnits(await token.totalSupply(), 18));

await (await token.transfer(alice.address, ethers.parseUnits("250", 18))).wait();
console.log("owner balance:", ethers.formatUnits(await token.balanceOf(owner.address), 18));
console.log("alice balance:", ethers.formatUnits(await token.balanceOf(alice.address), 18));

await (await token.approve(alice.address, ethers.parseUnits("100", 18))).wait();
console.log("allowance:", ethers.formatUnits(await token.allowance(owner.address, alice.address), 18));

const asAlice = token.connect(alice);
await (await asAlice.transferFrom(owner.address, alice.address, ethers.parseUnits("40", 18))).wait();
console.log("alice after transferFrom:", ethers.formatUnits(await token.balanceOf(alice.address), 18));

await (await token.mint(alice.address, ethers.parseUnits("10", 18))).wait();
await (await asAlice.burn(ethers.parseUnits("5", 18))).wait();
console.log("alice after mint+burn:", ethers.formatUnits(await token.balanceOf(alice.address), 18));
console.log("totalSupply after burn:", ethers.formatUnits(await token.totalSupply(), 18));
```

**output**

```
name: Study Token
symbol: STUDY
decimals: 18
totalSupply: 1000.0
owner balance: 750.0
alice balance: 250.0
allowance: 100.0
alice after transferFrom: 290.0
alice after mint+burn: 295.0
totalSupply after burn: 1005.0
```

Every log line is a real read or write: notify the decimals before formatting, and remember that `formatUnits` prints a readable number while the contract holds raw integers.

Example 3: the events the run produced, read back from the chain.

**JavaScript**

```javascript
const logs = await token.queryFilter("Transfer", 0, "latest");
console.log("Transfer events:", logs.length);
for (const log of logs) console.log("  ", log.args.map(String));

try {
  await asAlice.mint(alice.address, 1n);
  console.log("ERROR: a non-owner minted");
} catch (e) {
  console.log("non-owner mint reverts:", e.shortMessage ?? e.message);
}
```

**output**

```
Transfer events: 5
   [
  '0x0000000000000000000000000000000000000000',
  '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  '10000000000000000000'
]
   [
  '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  '0x70997970C51812dc3A010C7d01b50e0d17dc79C8',
  '25000000000000000000'
]
   [
  '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
  '0x70997970C51812dc3A010C7d01b50e0d17dc79C8',
  '40000000000000000000'
]
   [
  '0x0000000000000000000000000000000000000000',
  '0x70997970C51812dc3A010C7d01b50e0d17dc79C8',
  '10000000000000000000'
]
   [
  '0x70997970C51812dc3A010C7d01b50e0d17dc79C8',
  '0x0000000000000000000000000000000000000000',
  '50000000000000000000'
]
non-owner mint reverts: execution reverted (unknown custom error)
```

A mint appears as a transfer from the zero address and a burn as a transfer to it, which is how every explorer labels them. The `unknown custom error` message is OpenZeppelin rejecting the non-owner mint with a typed error that the token's own ABI does not describe.

Example 4: an ERC-721 collection whose metadata points at IPFS.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC721} from "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import {ERC721URIStorage} from "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract StudyNFT is ERC721URIStorage, Ownable {
    uint256 private nextTokenId;

    constructor() ERC721("Study NFT", "SNFT") Ownable(msg.sender) {}

    function mint(address to, string calldata tokenURI_)
        external
        onlyOwner
        returns (uint256)
    {
        uint256 tokenId = nextTokenId++;
        _safeMint(to, tokenId);
        _setTokenURI(tokenId, tokenURI_);
        return tokenId;
    }
}
```

**JavaScript**

```javascript
// after deploying, against the same local node
await (await nft.mint(owner.address, "ipfs://bafy1/metadata/1.json")).wait();

console.log("ownerOf(0):", await nft.ownerOf(0));
console.log("tokenURI(0):", await nft.tokenURI(0));
await (await nft.transferFrom(owner.address, alice.address, 0)).wait();
console.log("owner after transfer:", await nft.ownerOf(0));
console.log("Transfer events:", (await nft.queryFilter("Transfer", 0, "latest")).length);
```

**output**

```
ownerOf(0): 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
tokenURI(0): ipfs://bafy1/metadata/1.json
owner after transfer: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
Transfer events: 2
```

The two events are the mint, from the zero address, and the transfer. A real collection would host the metadata JSON through a gateway and its `image` field would be another `ipfs://` URI, which is the off-chain half described in chapter 9.

Example 5: an ERC-1155 contract for a game's item families.

**Solidity**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC1155} from "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";

contract StudyItems is ERC1155 {
    uint256 public constant SWORD = 0;
    uint256 public constant SHIELD = 1;

    constructor() ERC1155("https://study.example/item/{id}.json") {}

    function mintBatch(address to, uint256[] memory ids, uint256[] memory amounts)
        external
    {
        _mintBatch(to, ids, amounts, "");
    }
}
```

**JavaScript**

```
await (await items.mintBatch(alice.address, [0, 1], [10, 5])).wait();

console.log("alice sword balance:", (await items.balanceOf(alice.address, 0)).toString());
console.log("alice shield balance:", (await items.balanceOf(alice.address, 1)).toString());
```

**output**

```
alice sword balance: 10
alice shield balance: 5
```

One contract, two item types, one transaction. `uri(0)` would replace `{id}` in the template with the id when a wallet asks for metadata.

## 7.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Forking a token contract instead of using a library | you own every bug | import OpenZeppelin, extend it |
| Granting unlimited allowance to an unknown site | the site can drain balances | approve the exact amount, and revoke when done |
| `mint` open to everyone | runaway supply | gate it behind a role, a cap, or a rule |
| `transferFrom` without safe checks in an NFT flow | tokens sent to contracts that cannot hold them | use `safeTransferFrom` |
| Storing image data on-chain | deployment cost explodes | store a content hash, host the file elsewhere |
| Assuming decimals are always 18 | display and math break | read `decimals()` and never hardcode |

- Treat the constructor as policy: the supply rule, the owner, and the metadata base are all decided there.

- Before sending a token transaction, read the allowance you are granting and say it out loud in the UI.

## 7.5 Practice exercises

1. Which two functions does a spender use to move your ERC-20 tokens?
2. What does a transfer from the zero address mean in an ERC-20 log?
3. Why can a plain ERC-721 `transferFrom` strand a token?
4. In ERC-1155, how does the `uri` template resolve for id 7?
5. Which three properties does the constructor of `StudyToken` decide?

### Solutions

1. The owner first calls `approve`, then the spender calls `transferFrom`.
2. That the tokens were minted.
3. Because it skips the receiver check, so a contract that does not implement the receiver hook accepts the token silently and may never move it again.
4. The template's `{id}` becomes the id in lowercase hex, so id 7 resolves the placeholder to `0x7` inside the URI.
5. The name and symbol, the initial supply and its recipient, and the initial owner through the `Ownable` constructor.

## 7.6 Mini test

1. Multiple choice: which standard is fungible? - a) ERC-721 - b) ERC-1155 - c) ERC-20 - d) ERC-165
2. True or false: `decimals()` is guaranteed to return 18.
3. Multiple choice: which function lets a market move your ERC-20 tokens? - a) `transfer` - b) `transferFrom`, after `approve` - c) `safeTransferFrom` - d) `burn`
4. Short answer: which address appears as the sender of a minting `Transfer` log?
5. True or false: ERC-1155 can hold both fungible and non-fungible ids in one contract.
6. Multiple choice: what does `safeTransferFrom` check that `transferFrom` does not? - a) the gas price - b) that the recipient can receive the token - c) the nonce - d) the owner's balance
7. Short code question: name the function that caps who may mint in `StudyToken`.

### Answer key and explanations

1. c - ERC-20 is the fungible standard.
2. False - 18 is common and OpenZeppelin's default, but it is an author choice.
3. b - approval plus `transferFrom` is the delegation pair.
4. The zero address, `0x000...000`, by convention.
5. True - one contract can manage many id families with any divisibility.
6. b - it verifies the recipient contract can handle the token, preventing stranded assets.

7. `onlyOwner`, from OpenZeppelin's `Ownable`.

# Chapter 8 - Building a dApp frontend

## 8.1 Summary

- A dApp is a normal frontend whose state-changing actions are transactions instead of fetch calls.
- Libraries split into two roles: a public client that reads the chain and a wallet client that signs and sends.
- A browser wallet is an EIP-1193 provider: JavaScript asks it for accounts, signatures, and transactions, and the user approves each one.
- Connecting is a read-only introduction; a transaction is the act that costs gas and needs a decision.
- The safe flow is simulate, send, then wait for the receipt and confirm the status.
- Revert data arrives as an encrypted-looking hex string, and libraries decode it against the ABI or fall back to a generic message.
- Two unwritten rules: never block the UI on a transaction, and never keep a key in the frontend.

## 8.2 Concepts

### Two clients, two jobs

**output**

```
frontend                      wallet (keys stay here)
   |                                  ^
   |  read:  publicClient / provider  |
   +--------------------> node        |
   |                                  |
   |  write: walletClient / signer ---> signature -> node
```

The public client answers questions: chain ID, block number, balances, contract reads. The wallet client prepares transactions and asks the wallet to sign them. Because keys live in the wallet, the frontend never handles one.

### The connection sequence

1. The page loads and creates a public client bound to a chain, such as localhost or Sepolia.
2. The user clicks connect; the frontend calls `eth_requestAccounts`.
3. The wallet shows the origin of the request and the user approves.
4. The frontend stores the account address in memory and re-renders.
5. Reads happen continuously; writes happen on user actions only.

A read-only call is `eth_call`: it executes against a local copy of the state and cannot change anything. A write is a signed transaction that costs gas.

### The lifecycle of a write

**output**

```
click      -> simulate  -> send     -> pending in the mempool
   -> included in a block        -> receipt with status 1 or 0
   -> UI confirms                -> optionally wait N more blocks
```

Waiting for one inclusion is usually enough on a test or local chain. On mainnet many teams wait a few blocks before treating large movements as final.

## 8.3 Examples

Example 1: reading the chain with viem. This is the exact script used to verify this volume, run against a local node, with a compiled counter artifact.

**JavaScript**

```javascript
import { readFileSync } from "node:fs";
import {
  createPublicClient,
  createWalletClient,
  http,
  parseEther,
  formatEther,
  getAddress,
} from "viem";
import { privateKeyToAccount } from "viem/accounts";
import { hardhat } from "viem/chains";

const artifact = JSON.parse(
  readFileSync("./artifacts/contracts/Counter.sol/Counter.json", "utf8"),
);

const account = privateKeyToAccount(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
);

const publicClient = createPublicClient({
  chain: hardhat,
  transport: http("http://127.0.0.1:8545"),
});

console.log("chainId:", await publicClient.getChainId());
console.log("blockNumber:", await publicClient.getBlockNumber());
console.log("balance:", formatEther(await publicClient.getBalance({ address: account.address })));
console.log("checksummed:", getAddress(account.address));
console.log("parseEther result:", parseEther("1.5"));

const walletClient = createWalletClient({
  account,
  chain: hardhat,
  transport: http("http://127.0.0.1:8545"),
});

const deployHash = await walletClient.deployContract({
  abi: artifact.abi,
  bytecode: artifact.bytecode,
});
const receipt = await publicClient.waitForTransactionReceipt({ hash: deployHash });
console.log("deploy status:", receipt.status);

const incHash = await walletClient.writeContract({
  address: receipt.contractAddress,
  abi: artifact.abi,
  functionName: "inc",
});
await publicClient.waitForTransactionReceipt({ hash: incHash });

const x = await publicClient.readContract({
  address: receipt.contractAddress,
  abi: artifact.abi,
  functionName: "x",
});
```

**JavaScript (continued)**

```
console.log("x:", x);

const logs = await publicClient.getContractEvents({
  address: receipt.contractAddress,
  abi: artifact.abi,
  eventName: "Incremented",
  fromBlock: 0n,
});
console.log("events:", logs.length);
console.log("event name:", logs[0].eventName);
console.log("event args by:", logs[0].args.by);
console.log("event args newValue:", logs[0].args.newValue);
```

**output**

```
chainId: 31337
blockNumber: 0n
balance: 10000
checksummed: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
parseEther result: 1500000000000000000n
deploy status: success
x: 1n
events: 1
event name: Incremented
event args by: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
event args newValue: 1n
```

viem works in native `bigint` end to end: the block number prints as `0n`, the ether value as `10000`, and the contract value as `1n`. That removes a whole class of float mistakes.

Example 2: the same script with ethers 6, for comparison.

**JavaScript**

```
import { readFileSync } from "node:fs";
import { ethers } from "ethers";

const artifact = JSON.parse(
  readFileSync("./artifacts/contracts/Counter.sol/Counter.json", "utf8"),
);

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const wallet = new ethers.Wallet(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
  provider,
);

console.log("chainId:", (await provider.getNetwork()).chainId.toString());
console.log("balance:", ethers.formatEther(await provider.getBalance(wallet.address)));

const counter = await new ethers.ContractFactory(
  artifact.abi,
  artifact.bytecode,
  wallet,
).deploy();
await counter.waitForDeployment();
console.log("deployed at:", await counter.getAddress());

const tx = await counter.inc();
const receipt = await tx.wait();
console.log("x:", (await counter.x()).toString());
console.log("gas used:", receipt.gasUsed.toString());
```

**output**

```
chainId: 31337
balance: 10000.0
deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3
x: 1
gas used: 45130
```

Both libraries end at the same state through different idioms: ethers exposes `BigInt` and `Result` objects, viem stays closer to raw hex and viem chains.

Example 3: a wallet connection in the browser. This code needs a page served over http or https and a wallet extension, so it was NOT executed while writing this volume; the shapes follow the EIP-1193 provider documentation.

**HTML**

```
<button id="connect">Connect wallet</button>
<pre id="out"></pre>
<script type="module">
  const out = document.getElementById("out");
  const eth = window.ethereum;
  if (!eth) {
    out.textContent = "No injected wallet found";
  } else {
    document.getElementById("connect").addEventListener("click", async () => {
      const [account] = await eth.request({ method: "eth_requestAccounts" });
      const chainId = await eth.request({ method: "eth_chainId" });
      out.textContent = `account ${account} on chain ${chainId}`;
    });
  }
</script>
```

Expected behaviour: clicking the button opens the wallet prompt; on approval the line shows an `0x` address and a hex chain ID such as `0x7a69` for a local chain. The wallet is the only component allowed to ask for approval, and the page must react to account and chain changes.

Example 4: simulate before sending. `simulateContract` runs the call against a state preview and returns the decoded result or throws the revert reason, so the user learns about a failure before paying for it.

**JavaScript**

```
const simulation = await publicClient.simulateContract({
  address: counterAddress,
  abi: counterAbi,
  functionName: "inc",
  account: account.address,
});
const hash = await walletClient.writeContract(simulation.request);
```

A void function such as `inc` simulates with no value, so `simulation.result` is `undefined`; a view function returns its value. If the call would revert, the simulate step throws and no transaction is built.

Example 5: showing the flow in a script with a real revert. The vault from chapter 11 rejects a withdrawal of more than the staked amount.

**JavaScript**

```javascript
try {
  const tooMuch = await publicClient.simulateContract({
    address: vaultAddress,
    abi: vaultAbi,
    functionName: "withdraw",
    args: [parseEther("1000")],
    account: account.address,
  });
  await walletClient.writeContract(tooMuch.request);
} catch (error) {
  console.log("rejected before sending:", error.shortMessage ?? error.message);
}
```

**output**

```
rejected before sending: execution reverted: "Vault: not enough staked"
```

The verification run that produced that message used the same require string as the contract in chapter 11. Frontend code should surface the reason string and keep the rest of the page usable.

## 8.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Sending without simulating | users pay for failed transactions | simulate first, then send |
| Freezing the UI on a pending transaction | users click again and double-submit | disable the button, keep the page alive |
| Reading a stale account | actions signed by the wrong wallet | re-read the account on every write |
| Hardcoding a chain in the frontend | a wallet on another network bricks the page | read the chain and prompt to switch |
| Swallowing revert reasons | support cannot debug | log the decoded reason to a service |
| Loading the whole history on page load | slow, rate-limited queries | read summaries, fetch events for a view |

- Show the user the transaction hash as soon as it exists, and link to a block explorer.
- Never store a private key in `localStorage`, even for a demo.

## 8.5 Practice exercises

1. Which two client objects does viem give you, and what is each for?
2. Why is `eth_call` free and `eth_sendTransaction` not?
3. What does a receipt status of `0` mean?
4. Which provider method asks the wallet for accounts?
5. Why should a frontend simulate before sending?

### Solutions

1. A public client for reading the chain and a wallet client for signing and sending transactions.
2. `eth_call` executes against a copy of the state and changes nothing, so there is no inclusion cost; a transaction consumes block space and gas.
3. The transaction was included but reverted, so nothing it wrote took effect.
4. `eth_requestAccounts`.
5. Because a revert can be detected and shown to the user without paying gas, which saves money and frustration.

## 8.6 Mini test

1. Multiple choice: which object signs transactions in a browser? - a) the public client - b) the wallet - c) the RPC node - d) the block explorer
2. True or false: `eth_call` can change chain state.
3. Multiple choice: what does viem's `getContractEvents` return? - a) raw bytes only - b) decoded logs with named event arguments - c) a transaction receipt - d) an ABI fragment
4. Short answer: which field of a receipt tells success?
5. True or false: a frontend should keep the user's private key in memory to sign faster.
6. Multiple choice: what does `parseEther("1.5")` return? - a) 1.5 - b) 1500000000000000000n - c) 150 gwei - d) a string
7. Short code question: name the viem function that waits for a transaction receipt.

### Answer key and explanations

1. b - the wallet owns the keys and the approval UI.
2. False - a call is simulated locally.
3. b - it decodes the logs against the ABI, giving `args` by parameter name.
4. `status`, where 1 means success and 0 means a revert.
5. False - keys stay in the wallet; a frontend holding one is a breach.
6. b - ether values are integer wei, and viem returns a `bigint`.
7. `waitForTransactionReceipt`.

# Chapter 9 - Reading blockchain data

## 9.1 Summary

- Logs are the cheap way a contract publishes facts: an event writes a record that off-chain code can search.

- A log has an address, up to three indexed topics, and one data blob, and `topics[0]` is always the keccak256 hash of the event signature.
- Marked `indexed` parameters become filterable topics; the rest are packed into the data field and must be decoded.
- Libraries wrap `eth_getLogs` and decode for you, but the filter shape is the same.
- Reading events is how you build balances, histories and notifications without keeping your own database in sync by hand.
- The Graph turns a contract's events into a queryable GraphQL API, called a subgraph, and is the standard answer to "how do I list everything".
- IPFS stores files by content hash, so a link proves the bytes and the file can be served by anyone.
- The common architecture is: contracts hold balances and rules, events hold the history, IPFS holds the files, and a subgraph holds the indexes.

## 9.2 Concepts

### The anatomy of a log

**output**

```
log record
  address      the contract that emitted it
  topics[0]    keccak256("Transfer(address,address,uint256)") = 0xddf252ad...
  topics[1..3] up to three indexed arguments
  data         the non-indexed arguments, ABI-encoded
  blockNumber, transactionHash, logIndex
```

Filtering by `topics[0]` alone finds every Transfer across all ERC-20 contracts; adding `topics[1]` narrows to one sender. This is why chapter 2's transfer hash matters.

### Two fetching styles

| Style | Method shape | Best for |
|---|---|---|
| Contract events | filter by event name and ABI | one contract's history |
| Raw logs | `eth_getLogs` with topic filters | a class of events across contracts |
| View calls | `eth_call` to getters | current values, no history |

A getter tells you the state now; a log tells you what happened. Dashboards need both.

### What The Graph does

**output**

```
contract events --> indexing nodes (subgraph) --> GraphQL endpoint
```

A subgraph is described by three files: a manifest saying which contract events to follow, a schema of the entities to store, and mappings that transform events into entity writes. Once deployed, querying is ordinary GraphQL with cursor pagination. The exact CLI and hosting commands change over time, so the block below marks what was and was not executed here.

### What IPFS does

IPFS addresses content by hash. You add a file, get a content identifier, and the network serves it from any peer that has it. Pinning is the promise to keep a copy available. NFTs typically store the metadata JSON on IPFS and put the `ipfs://` URI on-chain.

## 9.3 Examples

Example 1: reading a contract's events with ethers, against the counter from chapter 6.

**JavaScript**

```
const logs = await counter.queryFilter("Incremented", 0, "latest");
console.log("events:", logs.length);
for (const log of logs) {
  console.log(log.args.map(String), "block", log.blockNumber);
}
```

**output**

```
events: 1
[ '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266', '1' ] block 5
```

Example 2: the same logs with viem, which returns named arguments.

**JavaScript**

```
const logs = await publicClient.getContractEvents({
  address,
  abi: artifact.abi,
  eventName: "Incremented",
  fromBlock: 0n,
});
console.log("events:", logs.length);
console.log("args keys:", Object.keys(logs[0].args));
console.log("by:", logs[0].args.by);
console.log("newValue:", logs[0].args.newValue);
console.log("eventName:", logs[0].eventName);
```

**output**

```
events: 1
args keys: [ 'by', 'newValue' ]
by: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
newValue: 1
eventName: Incremented
```

Example 3: the raw RPC filter, for when no library fits. The response is a list of log objects, each with `topics` and `data` in hex.

**shell**

```
curl -s -X POST http://127.0.0.1:8545 \
  -H "Content-Type: application/json" \
  --data '{"jsonrpc":"2.0","method":"eth_getLogs","params":[{"fromBlock":"0x0","toBlock":"latest","topi
```

**JSON**

```
{"jsonrpc":"2.0","id":1,"result":[{"address":"0x...","topics":["0xddf252ad1be2c89b69c2b068fc378daa952ba
```

Only the filter topics shown are guaranteed; the address and the sender and recipient topics depend on which contract emitted them, so the shape above is a template rather than a captured response.

Example 4: a subgraph that indexes ERC-20 transfers. This is the standard three-file layout. The Graph command line and hosting were NOT executed in this environment, so no output is shown.

**YAML**

```yaml
specVersion: 1.0.0
description: Indexes Transfer and Approval events of one token
schema:
  file: ./schema.graphql
dataSources:
  - kind: ethereum
    name: StudyToken
    network: sepolia
    source:
      address: "0x0000000000000000000000000000000000000000"
      abi: StudyToken
      startBlock: 1
    mapping:
      kind: ethereum/events
      apiVersion: 0.0.7
      language: wasm/assemblyscript
      entities:
        - Transfer
      abis:
        - name: StudyToken
          file: ./abis/StudyToken.json
      eventHandlers:
        - event: Transfer(indexed address,indexed address,uint256)
          handler: handleTransfer
      file: ./src/mapping.ts
```

The manifest names the contract address, the ABI, the block to start from, and which events to follow. Replace the zero address with your deployment.

Example 5: the matching mapping and schema, written in TypeScript-like AssemblyScript.

**TypeScript**

```typescript
// src/mapping.ts - AssemblyScript, not executed in this environment
import { Transfer } from "../generated/StudyToken/StudyToken";

export function handleTransfer(event: Transfer): void {
  // entity writes go through the generated store, for example:
  // let item = new TokenMovement(event.transaction.hash.toHex());
  // item.from = event.params.from;
  // item.to = event.params.to;
  // item.value = event.params.value;
  // item.save();
}
```

**GraphQL**

```
  # schema.graphql - not executed in this environment
type TokenMovement @entity {
  id: Bytes!
  from: Bytes!
  to: Bytes!
  value: BigInt!
  blockNumber: BigInt!
}
```

`id` must be unique per entity; the transaction hash plus a log index is the usual choice.

Example 6: putting a file on IPFS. The daemon was NOT running in the environment used to verify this volume, so the output is described, not invented.

**shell**

```
ipfs add study-nft.png
ipfs pin add <cid>
ipfs cat <cid> | head -c 32
```

Expected behaviour, in words: `ipfs add` prints a warning, then the content identifier of the file, something like `bafy...`, which is 32 to 60 characters of base32 and encodes the hash; `ipfs pin add` confirms the object; `ipfs cat` streams the bytes back. A browser then reads the file through a gateway URL of the form `https://ipfs.io/ipfs/<cid>`. Gateways can be slow, so many projects pin on more than one service or run their own node.

## 9.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Querying logs from block 0 every page load | the range grows forever | cache the indexed height |
| Ask for all events of a busy token | rate limits and timeouts | filter by address and topic where possible |
| Emitting events without indexing the useful field | clients scan everything | index the columns you filter on |
| Storing whole documents on-chain | deployment and call costs | store a content hash, host the file |
| Assuming a gateway link proves ownership | the bytes prove content, not rights | verify the on-chain pointer and the owner |
| Relying on `topics` order beyond the first three | only three slots exist | pack the rest into data |

- One entity per indexed question: if your UI needs a query, the subgraph needs the entity that answers it.
- When a log query is slow, narrow the block range first; it is almost always the range, not the node.

## 9.5 Practice exercises

1. What is `topics[0]` of every Transfer log?
2. Why can at most three event parameters be indexed?
3. When should you prefer a view call over a log query?
4. What does pinning an IPFS file guarantee?
5. Why is the sender topic of a raw log filter not enough to find one account's history?

### Solutions

1. The keccak256 hash of the event signature, here `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`.
2. Because the log structure reserves exactly three slots after the signature topic.
3. When you need the current value of state rather than the history of changes.
4. That at least one service keeps a copy available; without a pin, a node may garbage-collect the file and the CID becomes unresolvable even though it is still a valid hash.
5. Because a sender topic finds transfers out of the account, not into it; a full history needs the recipient topic too.

## 9.6 Mini test

1. Multiple choice: which hash is `topics[0]` for `Transfer(address,address,uint256)`? - a) the sender address - b) the keccak256 of the signature - c) the block hash - d) the contract address
2. True or false: non-indexed parameters can be filtered directly.
3. Multiple choice: which of these does a subgraph NOT provide? - a) GraphQL queries - b) entity storage built from events - c) a place to keep private keys - d) an indexing manifest
4. Short answer: how many indexed topic slots does a log have after the signature?
5. True or false: an IPFS CID identifies content rather than a location.
6. Multiple choice: which filter field narrows a raw log query to one emitting contract? - a) `fromBlock` - b) `address` - c) `topics[0]` - d) `id`
7. Short code question: with ethers, fetch all `Incremented` logs of a contract.

### Answer key and explanations

1. b - the signature hash, which is the same for every ERC-20.
2. False - non-indexed arguments live in `data` and must be decoded.
3. c - a subgraph is an index of public data and holds no secrets.
4. Three.
5. True - the content identifier is the hash of the bytes.
6. b - `address` restricts to logs emitted by that contract.
7. `await contract.queryFilter("Incremented", 0, "latest");`

# Chapter 10 - Security, patterns, and upgradeability

## 10.1 Summary

- Reentrancy is the classic Ethereum bug: a contract calls an attacker while its state still says the attacker deserves money.
- The checks-effects-interactions order defeats it by updating state before any external call.
- A reentrancy guard adds a lock that reverts a second entry in the same call.
- `tx.origin` authorization is dangerous because a victim contract can call you with their credentials.
- Most access control is a constructor-set owner plus a modifier, and every privileged function needs that check.
- Unbounded loops can make a contract that works on day one fail on day one hundred.
- Since solc 0.8 arithmetic is checked, so overflow is a solved class, but compiler switches and assembly re-open it.
- Upgradeable contracts keep code in an implementation and state in a proxy joined by `delegatecall`, with storage layout compatibility as the price.

## 10.2 Concepts

### How the attack runs

**output**

```
1. victim deposits 3 ETH             Bank.balances[victim] = 3
2. attacker deposits 1 ETH           Bank.balances[attacker] = 1
3. attacker calls withdraw()
     Bank pays 1 ETH to the attacker contract
       -> attacker receive() runs while Bank.balances is STALE
       -> attacker calls withdraw() again, and the bank pays again
4. bank is drained
```

The fix is not a secret: set the balance to zero before sending, and lock the function so a second entry reverts.

### The vulnerability map

| Family | One-line description | Main defense |
|---|---|---|
| Reentrancy | external call re-enters before state settles | checks-effects-interactions, guard |
| Access control | a privileged function forgot the owner check | explicit owner or role check |
| Front-running | a pending transaction is copied with a higher fee | commit-reveal, tight slippage limits |

| Family | One-line description | Main defense |
|---|---|---|
| Timestamp dependence | the block time is slightly gameable | avoid exact-time decisions |
| Denial of service | unbounded loops or griefing by anyone | caps, pull payments |
| Oracle manipulation | price taken from a swimmable pool | time-weighted or decentralized oracles |
| Delegatecall abuse | storage and code mix unexpectedly | well-audited proxy libraries |
| Signature replay | the same signature used twice | nonces in the signed payload |

### Patterns that pay for themselves

- Pull over push. Let users withdraw rather than paying them in a loop: one failing recipient cannot block everyone.
- Circuit breaker. A pause switch owned by governance stops the bleeding while a bug is fixed.
- Spending limits. A daily cap on what a privileged role can move.
- Two-step ownership handover. The new owner accepts, so a typo does not brick the contract.

### Proxies without magic

**output**

```
user ---> PROXY (holds the storage) --delegatecall--> IMPLEMENTATION (holds the code)
             ERC-1967 slot stores the implementation address
```

`delegatecall` runs the implementation's code against the proxy's storage, so the state survives code upgrades. Two popular layouts: the transparent proxy, where an admin cannot call implementation functions, and UUPS, where the upgrade authorization lives in the implementation itself. Both demand that the new implementation keeps the storage layout of the old one: never reorder, never repurpose, only append.

## 10.3 Examples

Example 1: the guarded bank, and the attacker that proves the guard works.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract Bank is ReentrancyGuard {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external nonReentrant {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "Bank: nothing to withdraw");
        balances[msg.sender] = 0;                  // effects before interactions
        (bool ok, ) = msg.sender.call{value: amount}("");
        require(ok, "Bank: transfer failed");
    }

    function depositOf(address who) external view returns (uint256) {
        return balances[who];
    }
}

contract Attacker {
    Bank public bank;
    uint256 public reentries;

    constructor(Bank bank_) {
        bank = bank_;
    }

    function attack() external payable {
        bank.deposit{value: msg.value}();
        bank.withdraw();
    }

    receive() external payable {
        if (address(bank).balance >= 1 ether) {
            reentries += 1;
            try bank.withdraw() {} catch {}         // the retry reverts, safely
        }
    }
}
```

Example 2: the run. A victim deposits 3 ETH, the attacker attacks with 1 ETH, and the guard stops the second withdrawal.

**JavaScript**

```ts
import { readFileSync } from "node:fs";
import { ethers } from "ethers";

const bankArtifact = JSON.parse(
  readFileSync("./artifacts/contracts/Bank.sol/Bank.json", "utf8"),
);
const attackerArtifact = JSON.parse(
  readFileSync("./artifacts/contracts/Bank.sol/Attacker.json", "utf8"),
);

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const victimWallet = new ethers.Wallet(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
  provider,
);
const attackerWallet = new ethers.Wallet(
  "0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d",
  provider,
);
const attacker = new ethers.NonceManager(attackerWallet);

const bank = await new ethers.ContractFactory(
  bankArtifact.abi,
  bankArtifact.bytecode,
  attacker,
).deploy();
await bank.waitForDeployment();

const attackerContract = await new ethers.ContractFactory(
  attackerArtifact.abi,
  attackerArtifact.bytecode,
  attacker,
).deploy(await bank.getAddress());
await attackerContract.waitForDeployment();

await (await bank.connect(victimWallet).deposit({ value: ethers.parseEther("3") })).wait();
console.log("bank balance:", ethers.formatEther(await provider.getBalance(await bank.getAddress())));
console.log("victim deposit recorded:", ethers.formatEther(await bank.depositOf(victimWallet.address)));

await (await attackerContract.connect(attacker).attack({ value: ethers.parseEther("1") })).wait();
console.log("reentry attempts seen:", (await attackerContract.reentries()).toString());
console.log("bank balance after attack:", ethers.formatEther(await provider.getBalance(await bank.getAd
console.log("victim still holds:", ethers.formatEther(await bank.depositOf(victimWallet.address)));

try {
  await bank.connect(attackerWallet).withdraw();
  console.log("ERROR: the attacker withdrew the victim deposit");
} catch (e) {
  console.log("attacker cannot withdraw victim deposit:", e.shortMessage ?? e.message);
}

await (await bank.connect(victimWallet).withdraw()).wait();
console.log("bank left after victim withdraw:", ethers.formatEther(await provider.getBalance(await bank
```

**output**

```
bank balance: 3.0
victim deposit recorded: 3.0
reentry attempts seen: 1
bank balance after attack: 3.0
victim still holds: 3.0
attacker cannot withdraw victim deposit: execution reverted: "Bank: nothing to withdraw"
bank left after victim withdraw: 0.0
```

The attacker's `receive` did fire and did call `withdraw` again: the counter proves the reentry happened, and the unchanged balance proves the lock stopped it. The attacker walks away with only their own ether.

Example 3: an upgradeable pair and the proxy that holds the state.

**Solidity**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Initializable} from "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import {OwnableUpgradeable} from "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
import {UUPSUpgradeable} from "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

contract StudyV1 is Initializable, OwnableUpgradeable, UUPSUpgradeable {
    uint256 public value;

    function initialize() public initializer {
        __Ownable_init(msg.sender);
    }

    function setValue(uint256 newValue) external onlyOwner {
        value = newValue;
    }

    function _authorizeUpgrade(address newImplementation)
        internal
        override
        onlyOwner
    {}
}

contract StudyV2 is StudyV1 {
    uint256 public doubled;

    function doubleValue() external onlyOwner {
        doubled = value * 2;
    }
}
```

Note the transport: version 5 of the upgradeable library has no empty `__UUPSUpgradeable_init()` to call, so the initializer only sets ownership, and the proxy points at a constructor argument.

Example 4: deploying V1 behind a proxy, calling through it, then upgrading.

**JavaScript**

```
import { readFileSync } from "node:fs";
import { ethers } from "ethers";

const v1Artifact = JSON.parse(
  readFileSync("./artifacts/contracts/StudyV1.sol/StudyV1.json", "utf8"),
);
const v2Artifact = JSON.parse(
  readFileSync("./artifacts/contracts/StudyV1.sol/StudyV2.json", "utf8"),
);
const proxyArtifact = JSON.parse(
  readFileSync("./artifacts/contracts/StudyProxy.sol/StudyProxy.json", "utf8"),
);

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const deployer = new ethers.NonceManager(new ethers.Wallet(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
  provider,
));

const v1 = await new ethers.ContractFactory(v1Artifact.abi, v1Artifact.bytecode, deployer).deploy();
await v1.waitForDeployment();
console.log("impl value before initialization:", (await v1.value()).toString());

const initData = v1.interface.encodeFunctionData("initialize");
const proxy = await new ethers.ContractFactory(proxyArtifact.abi, proxyArtifact.bytecode, deployer)
  .deploy(await v1.getAddress(), initData);
await proxy.waitForDeployment();

const throughProxy = new ethers.Contract(await proxy.getAddress(), v1Artifact.abi, deployer);
console.log("proxy owner:", await throughProxy.owner());
await (await throughProxy.setValue(21)).wait();
console.log("value through proxy:", (await throughProxy.value()).toString());

const v2 = await new ethers.ContractFactory(v2Artifact.abi, v2Artifact.bytecode, deployer).deploy();
await v2.waitForDeployment();
await (await throughProxy.upgradeToAndCall(await v2.getAddress(), "0x")).wait();

const v2Read = new ethers.Contract(await proxy.getAddress(), v2Artifact.abi, deployer);
await (await v2Read.doubleValue()).wait();
console.log("doubled through upgraded proxy:", (await v2Read.doubled()).toString());
```

**output**

```
impl value before initialization: 0
proxy owner: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
value through proxy: 21
doubled through upgraded proxy: 42
```

Two details worth noticing. The implementation, read directly, shows the uninitialized storage, because all real state lives in the proxy. And the owner survives the upgrade, because ownership is stored in the proxy's storage slot and the new code only reads it.

Example 5: verifying the upgrade is real, and that only the owner may do it.

**JavaScript**

```
const ERC1967_SLOT = "0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc";
const slot = await provider.getStorage(await proxy.getAddress(), ERC1967_SLOT);
const v2Address = await v2.getAddress();
console.log("slot points at V2:", slot.toLowerCase() === "0x000000000000000000000000" + v2Address.slice

const stranger = new ethers.Wallet(
  "0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a",
  provider,
);
try {
  await throughProxy.connect(stranger).upgradeToAndCall(v2Address, "0x");
  console.log("ERROR: a stranger upgraded the proxy");
} catch (e) {
  console.log("stranger cannot upgrade:", e.shortMessage ?? e.message);
}
```

**output**

```
slot points at V2: true
stranger cannot upgrade: execution reverted (unknown custom error)
```

The ERC-1967 slot is a fixed, well-known storage address that points at the implementation. Reading it directly is the ground truth for what a proxy runs.

## 10.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| External call before state update | reentrancy drains the contract | checks-effects-interactions |
| Only `transfer` or `send` for ETH | 2300 gas can break the receiver | `call` with a guard, or pull payments |
| Same name in two base contracts without resolve | compile error or wrong logic | explicit `override(Base1, Base2)` |
| Changing storage layout across an upgrade | the proxy's state is misread | append only, and test the upgrade path |
| Unnumbered, uncapped loops | the contract freezes as data grows | caps plus pagination |
| Reusing a test private key in a config | accidental live deployment | separate configs, one network each |
| Assuming an audit means safe | audits are point-in-time reviews | run a bug bounty and keep monitoring |

- Write the exploit script before you need it: a test that attempts the attack is the only proof a defense works.
- For upgrades, keep a migration test that deploys old state, upgrades, and asserts the invariants still hold.

## 10.5 Practice exercises

1. Put the three steps of a `withdraw` function in the safe order.
2. Why does `nonReentrant` not by itself fix the CEI violation?
3. What does `delegatecall` preserve, and what does it share?
4. Why must an implementation contract never have a constructor that sets state?
5. Which storage slot holds a modern proxy's implementation address?

### Solutions

1. Checks, then effects, then the external interaction: validate inputs, update balances, and only then transfer.
2. Because the lock only stops a re-entry that happens within the same call; a contract that is dirty in other ways, or reentered across transactions, still loses. Defense in depth is the point.
3. `delegatecall` runs the implementation's code in the caller's context, so storage, `msg.sender` and address belong to the proxy, and the state persists in the proxy.
4. Because a constructor runs only in the implementation's own storage, which nobody uses; the equivalent work belongs in an `initialize` function guarded by `initializer`.
5. The ERC-1967 implementation slot, a fixed keccak-derived slot at `0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc`.

## 10.6 Mini test

1. Multiple choice: which order defeats reentrancy? - a) interactions, effects, checks - b) checks, effects, interactions - c) effects, interactions, checks - d) there is no safe order
2. True or false: `tx.origin` is a safe place to check ownership.
3. Multiple choice: what does a proxy store that the implementation does not? - a) code - b) the state - c) events - d) the compiler version
4. Short answer: which OpenZeppelin modifier blocks a second entry?
5. True or false: since solc 0.8, arithmetic cannot overflow at all.
6. Multiple choice: which pattern moves payment from the contract to a user-triggered claim? - a) push payments in a loop - b) pull payments - c) batch transfer - d) mining rewards
7. Short code question: in one line, mark an upgradeable initializer so it can run only once.

### Answer key and explanations

1. b - checks, effects, interactions.
2. False - `tx.origin` is the transaction's ultimate caller and enables phishing via an intermediate contract.
3. b - the proxy holds the storage; the implementation holds the code.
4. `nonReentrant`.
5. False - `unchecked`, inline assembly and libraries can still wrap.
6. b - pull payments put the claim in the user's hands.

7. `function initialize() public initializer {`

# Chapter 11 - DeFi and DAO building blocks

## 11.1 Summary

- DeFi is the set of composable primitives built from tokens and contracts: exchanges, lending markets, and staking vaults.
- An automated market maker prices assets with a constant product formula instead of an order book.
- Liquidity providers deposit both sides of a pool and earn fees, at the cost of impermanent loss when the price moves.
- Slippage is the gap between the quoted and executed price, and it grows with trade size in shallow pools.
- Oracles deliver prices to contracts, and a manipulable pool makes a terrible oracle.
- Lending is overcollateralized: value is borrowed against value, and positions fall below a health threshold can be liquidated by anyone.
- A DAO is a contract that collects proposals and votes, and executes what the votes decide, with a timelock between the decision and the effect.
- The security lessons from chapter 10 apply with full force, because these contracts hold money in public.

## 11.2 Concepts

### Constant product

A pool holds reserves X and Y and keeps the invariant X * Y = k. Buying X with an amount dx shifts the reserves, and the contract pays out `dy` such that

**output**

```
(X + dx) * (Y - dy) = k     =>     dy = Y - k / (X + dx)
```

The bigger the trade relative to the pool, the worse the effective price, which is slippage. A 0.3% fee rewards the pool and makes the invariant grow.

| Term | Meaning |
|---|---|
| Liquidity provider | deposits X and Y, receives a pool token as receipt |
| Impermanent loss | the value of the pair lags holding the two assets apart |
| Slippage | execution price versus quoted price |
| Fee tier | the fraction of each trade that stays in the pool |

### Lending and liquidation

A borrower locks collateral worth more than the loan. If the collateral value falls and the loan approaches the cap, the health factor crosses a threshold, and any keeper can repay part of the debt and seize the collateral at a discount. That's liquidation, and it is how a lending market stays solvent without a legal system.

### Governance

**output**

```
propose -> vote -> queue (timelock) -> execute
```

A governor contract holds the rules: who may propose, how long voting lasts, what share of tokens makes a quorum, and the threshold to pass. The timelock delays execution so users can exit before a decision takes effect.

## 11.3 Examples

Example 1: the swap formula in plain JavaScript. No library, no chain, so this runs under plain Node exactly as written.

**JavaScript**

```
const tokenX = 100_000n * 10n ** 18n; // 100,000 X in the pool
const tokenY = 50_000n * 10n ** 18n; // 50,000 Y in the pool
const k = tokenX * tokenY;

function price() {
  return Number(tokenY * 10n ** 18n / tokenX) / 10 ** 18;
}
function inY(wei) {
  return Number(wei) / 10 ** 18;
}

console.log("initial price of 1 X in Y:", price().toFixed(6));

const dx = 1_000n * 10n ** 18n; // trader puts in 1,000 X
const dy = tokenY - k / (tokenX + dx);
console.log("trader receives Y:", inY(dy).toFixed(4));

const feeRateBps = 30n; // 0.30%
const fee = (dx * feeRateBps) / 10_000n;
const effectiveIn = dx - fee;
const dyWithFee = tokenY - k / (tokenX + effectiveIn);
console.log("trader receives Y with 0.3% fee:", inY(dyWithFee).toFixed(4));
console.log("pool keeps fee in X:", inY(fee).toFixed(4));
```

**output**

```
initial price of 1 X in Y: 0.500000
trader receives Y: 495.0495
trader receives Y with 0.3% fee: 493.5790
pool keeps fee in X: 3.0000
```

A thousand X buys a little under five hundred Y, not exactly five hundred, because the reserves shifted as the trade executed.

Example 2: a staking vault that pays a fixed reward rate per second.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

contract StakingVault {
    using SafeERC20 for IERC20;

    IERC20 public immutable stakeToken;
    IERC20 public rewardToken;

    uint256 public accRewardPerShare;
    uint256 public lastRewardTime;
    uint256 public rewardRatePerSecond;
    uint256 public totalStaked;

    struct UserInfo {
        uint256 staked;
        uint256 rewardDebt;
    }

    mapping(address => UserInfo) public users;

    event Staked(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event RewardPaid(address indexed user, uint256 amount);

    constructor(IERC20 stakeToken_, IERC20 rewardToken_, uint256 rewardRatePerSecond_) {
        stakeToken = stakeToken_;
        rewardToken = rewardToken_;
        rewardRatePerSecond = rewardRatePerSecond_;
        lastRewardTime = block.timestamp;
    }

    function pendingReward(address account) public view returns (uint256) {
        UserInfo storage user = users[account];
        uint256 accumulated = accRewardPerShare;
        if (totalStaked > 0) {
            accumulated +=
                (rewardRatePerSecond * (block.timestamp - lastRewardTime) * 1e18) /
                totalStaked;
        }
        return (user.staked * accumulated) / 1e18 - user.rewardDebt;
    }

    modifier updateReward(address account) {
        if (totalStaked > 0) {
            accRewardPerShare +=
                (rewardRatePerSecond * (block.timestamp - lastRewardTime) * 1e18) /
                totalStaked;
        }
        lastRewardTime = block.timestamp;
        if (account != address(0)) {
            uint256 pending = (users[account].staked * accRewardPerShare) / 1e18 -
```

**Solidity (continued)**

```
                users[account].rewardDebt;
            if (pending > 0) {
                users[account].rewardDebt =
                    (users[account].staked * accRewardPerShare) /
                    1e18;
                rewardToken.safeTransfer(account, pending);
                emit RewardPaid(account, pending);
            }
        }
        _;
    }

    function fund(uint256 amount) external {
        rewardToken.safeTransferFrom(msg.sender, address(this), amount);
    }

    function stake(uint256 amount) external updateReward(msg.sender) {
        require(amount > 0, "Vault: zero amount");
        users[msg.sender].staked += amount;
        totalStaked += amount;
        users[msg.sender].rewardDebt = (users[msg.sender].staked * accRewardPerShare) / 1e18;
        stakeToken.safeTransferFrom(msg.sender, address(this), amount);
        emit Staked(msg.sender, amount);
    }

    function withdraw(uint256 amount) external updateReward(msg.sender) {
        require(amount > 0, "Vault: zero amount");
        UserInfo storage user = users[msg.sender];
        require(user.staked >= amount, "Vault: not enough staked");
        user.staked -= amount;
        totalStaked -= amount;
        user.rewardDebt = (user.staked * accRewardPerShare) / 1e18;
        stakeToken.safeTransfer(msg.sender, amount);
        emit Withdrawn(msg.sender, amount);
    }
}
```

The accumulator pattern scales fixed-point rewards by 1e18 so that many small stakers and one big staker draw fairly; `rewardDebt` clears what has already been claimed.

Example 3: running the vault. The owner funds rewards, Alice stakes, time jumps one hundred seconds, and the pending reward matches the rate.

**JavaScript**

```
import { readFileSync } from "node:fs";
import { ethers } from "ethers";

const art = (name, c = name) =>
  JSON.parse(readFileSync(`./artifacts/contracts/${c}.sol/${name}.json`, "utf8"));

const tokenArtifact = art("StudyToken");
const vaultArtifact = art("StakingVault");

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const ownerWallet = new ethers.Wallet(
  "0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80",
  provider,
);
const aliceWallet = new ethers.Wallet(
  "0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d",
  provider,
);
const owner = new ethers.NonceManager(ownerWallet);
const alice = new ethers.NonceManager(aliceWallet);

const token = await new ethers.ContractFactory(tokenArtifact.abi, tokenArtifact.bytecode, owner)
  .deploy(ethers.parseUnits("10000", 18));
await token.waitForDeployment();
const vault = await new ethers.ContractFactory(vaultArtifact.abi, vaultArtifact.bytecode, owner)
  .deploy(await token.getAddress(), await token.getAddress(), ethers.parseUnits("1", 18));
await vault.waitForDeployment();

await (await token.transfer(aliceWallet.address, ethers.parseUnits("500", 18))).wait();
await (await token.mint(ownerWallet.address, ethers.parseUnits("5000", 18))).wait();

const tokenAsAlice = token.connect(alice);
await (await tokenAsAlice.approve(await vault.getAddress(), ethers.parseUnits("100", 18))).wait();
await (await vault.connect(alice).stake(ethers.parseUnits("100", 18))).wait();
console.log("alice staked:", ethers.formatUnits((await vault.users(aliceWallet.address)).staked, 18));

const tokenAsOwner = token.connect(owner);
await (await tokenAsOwner.approve(await vault.getAddress(), ethers.parseUnits("500", 18))).wait();
await (await vault.connect(owner).fund(ethers.parseUnits("500", 18))).wait();
console.log("vault balance (100 staked + 500 rewards):",
  ethers.formatUnits(await token.balanceOf(await vault.getAddress()), 18));

const nextTs = (await provider.getBlock("latest")).timestamp + 100;
await provider.send("evm_setNextBlockTimestamp", [nextTs]);
await provider.send("evm_mine", []);
console.log("pending reward after ~100s:", ethers.formatUnits(await vault.pendingReward(aliceWallet.add

await (await vault.connect(alice).withdraw(ethers.parseUnits("100", 18))).wait();
console.log("alice reward balance:", ethers.formatUnits(await token.balanceOf(aliceWallet.address), 18)
console.log("vault reward left:", ethers.formatUnits(await token.balanceOf(await vault.getAddress()), 1
```

**output**

```
alice staked: 100.0
vault balance (100 staked + 500 rewards): 600.0
pending reward after ~100s: 101.0
alice reward balance: 602.0
vault reward left: 398.0
```

At one whole reward token per second with a single staker, a hundred seconds yield a hundred tokens plus the rounding of the block that mines the withdrawal, which is why the line reads 101.0. Alice ends with her 500, minus the 100 she staked, plus the reward.

Example 4: a minimal DAO. Anyone may propose, addresses vote once, and execution is allowed only after the deadline.

**Solidity**

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract SimpleDAO {
    struct Proposal {
        string description;
        uint256 forVotes;
        uint256 againstVotes;
        uint256 deadline;
        bool executed;
        mapping(address => bool) hasVoted;
    }

    Proposal[] private proposals;
    uint256 public votingPeriod = 1 days;

    event Proposed(uint256 indexed id, string description, uint256 deadline);
    event Voted(address indexed voter, uint256 indexed id, bool support, uint256 votes);
    event Executed(uint256 indexed id);

    modifier proposalExists(uint256 id) {
        require(id < proposals.length, "DAO: unknown proposal");
        _;
    }

    function propose(string calldata description) external returns (uint256 id) {
        id = proposals.length;
        proposals.push();
        Proposal storage proposal = proposals[id];
        proposal.description = description;
        proposal.deadline = block.timestamp + votingPeriod;
        emit Proposed(id, description, proposal.deadline);
    }

    function vote(uint256 id, bool support) external proposalExists(id) {
        Proposal storage proposal = proposals[id];
        require(block.timestamp < proposal.deadline, "DAO: voting closed");
        require(!proposal.hasVoted[msg.sender], "DAO: already voted");
        proposal.hasVoted[msg.sender] = true;
        if (support) {
            proposal.forVotes += 1;
        } else {
            proposal.againstVotes += 1;
        }
        emit Voted(msg.sender, id, support, 1);
    }

    function state(uint256 id) public view returns (string memory) {
        if (proposals[id].executed) return "Executed";
        if (block.timestamp < proposals[id].deadline) return "Active";
        return proposals[id].forVotes > proposals[id].againstVotes ? "Passed" : "Rejected";
    }

    function execute(uint256 id) external proposalExists(id) {
        Proposal storage proposal = proposals[id];
        require(!proposal.executed, "DAO: already executed");
```

**Solidity (continued)**

```
        require(block.timestamp >= proposal.deadline, "DAO: voting open");
        require(proposal.forVotes > proposal.againstVotes, "DAO: proposal rejected");
        proposal.executed = true;
        emit Executed(id);
    }

    function proposalCount() external view returns (uint256) {
        return proposals.length;
    }
}
```

Example 5: running the DAO with a time jump past the deadline.

**JavaScript**

```
await (await dao.propose("fund the study group")).wait();
console.log("proposals:", (await dao.proposalCount()).toString());
await (await dao.connect(alice).vote(0, true)).wait();
await (await dao.connect(bob).vote(0, true)).wait();
console.log("state:", await dao.state(0));

const deadline = (await dao.getProposal(0))[3];
await provider.send("evm_setNextBlockTimestamp", [Number(deadline) + 1]);
await provider.send("evm_mine", []);
console.log("state after deadline:", await dao.state(0));
await (await dao.execute(0)).wait();
console.log("state after execute:", await dao.state(0));
```

**output**

```
proposals: 1
state: Active
state after deadline: Passed
state after execute: Executed
```

`getProposal` here is a small view helper returning the tuple of fields, because a public array of structs does not expose nested mappings.

## 11.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Trusting a pool's spot price as an oracle | a flash trade moves it | time-weighted or external oracles |
| No slippage bound on a swap | sandwiches eat the difference | a minimum-out parameter on every trade |
| Rewards that outrun the reserve | the last claimant cannot claim | cap emissions, or mint against a schedule |
| Votes counted per address, value held elsewhere | whale capture | decide weight rules on purpose |
| Execution before a delay | a passed vote can act instantly | a timelock the community can watch |

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Assuming an audited DeFi library is complete | you can compose it wrongly | read its threat model, then test yours |

- Model the money before the code: who pays in, who gets out, and what happens when the market moves 50% in a block.
- Any number a contract reads from the outside world is an oracle. Choose it deliberately.

## 11.5 Practice exercises

1. A pool holds 100,000 X and 50,000 Y. What is the price of one X in Y?
2. Why does a bigger trade produce worse slippage?
3. What is a health factor, in one sentence?
4. What does a timelock protect?
5. Why does the accumulator in the vault divide by total staked?

### Solutions

1. 0.5 Y per X, because the price is the ratio of the reserves.
2. Because the output formula grows the cost curve with the size of the input: each additional unit is priced against a pool that just became more unbalanced.
3. It is the ratio of collateral value to debt value: when it drops below one, the position can be liquidated.
4. It delays execution so holders and users can react before a passed proposal changes the contract or moves funds.
5. Because rewards are distributed in proportion to each account's share of the total stake, and the ratio must be known at every accrual.

## 11.6 Mini test

1. Multiple choice: which formula drives an automated market maker? - a) x + y = k - b) x * y = k - c) x / y = k - d) x - y = k
2. True or false: liquidity providers earn fees for bearing price risk.
3. Multiple choice: what is slippage? - a) a failed transaction - b) quoted versus executed price - c) the pool's fee - d) the gas price
4. Short answer: which price source is acceptable for a lending contract?
5. True or false: in this volume's DAO, voting is weighted by token balance.
6. Multiple choice: what happens when a collateralized debt falls below the health threshold? - a) nothing - b) anyone may liquidate it - c) the owner must intervene - d) the debt is forgiven
7. Short code question: in the vault, which function settles rewards and then runs the caller's body?

### Answer key and explanations

1. b - the constant product invariant.
2. True - fees compensate for impermanent loss risk.
3. b - slippage is execution price minus quote, driven by depth.
4. A time-weighted average or a decentralized oracle, never a shallow pool's spot price.
5. False - `SimpleDAO` counts one vote per address, which is why it is a teaching model rather than a token-weighted governor.
6. b - keepers may repay part of the debt and take the collateral at a discount, which keeps the market solvent.
7. `updateReward`, the modifier that accrues and pays before the body.

# Chapter 12 - Testing, verification, deploying, and beyond Ethereum

## 12.1 Summary

- A contract cannot be patched after deployment, so tests are the only quality gate you get before users are the testers.
- Hardhat tests run on an ephemeral in-process chain: deploy, act, assert, and repeat with clean state each time.
- Fork tests run against a copy of a public network's state, so real pool and token behavior can be exercised without spending money.
- The same test files run in Foundry as well, with its own runner, and this volume marks those commands as not executed here.
- Verification uploads the source to a block explorer so readers can read the contract, and it needs a live deployment plus an explorer API key.
- A deployment checklist is short: network, deployer, parameters, verification, monitoring, and a private runbook for the pause.
- Solana is the largest non-EVM ecosystem, with a different account and program model, its own token standard, and no direct compatibility with EVM code.
- Bridges connect chains by locking on one side and minting on the other, which makes the bridge itself the weakest link.

## 12.2 Concepts

### The four levels of testing

| Level | Environment | What it catches |
|---|---|---|
| Unit | in-process chain | logic and access-control bugs |
| Integration | local node plus clients | client wiring and event decoding |

| Level | Environment | What it catches |
|---|---|---|
| Fork | copy of a live network | interaction with real protocols |
| Invariant and fuzz | in-process chain with generated inputs | forgotten assumptions |

A fork test needs an archive endpoint, and it is slow enough that you keep a small number of them for the paths that touch other protocols.

## What a good test file contains

One test per rule, with an arrangement section at the top of each. Assert on return values, on state, and on events, because a contract can return the right number and still emit nothing. Use helpers such as `time.increaseTo` for deadlines and `impersonateAccount` when a call must come from another address.

## Before you deploy

- [ ] tests pass on a clean checkout
- [ ] the compiler version is fixed and the bytecode hash recorded
- [ ] constructor arguments are in a config file, not the command line
- [ ] the deployer key lives in the environment, and it is not the test key
- [ ] the network is spelled out and the chain ID is logged
- [ ] the verification step is ready to run
- [ ] the pause and ownership handover are documented

## Verification and explorers

Verification matches the deployed bytecode to the source you provide, so anyone can read the contract on a block explorer. The command needs the deployment address, the constructor arguments, and a network with an explorer API configured. Chapter 3's networks table names the explorers for each chain.

## Beyond the EVM

| Question | Ethereum | Solana |
|---|---|---|
| Execution model | EVM, sequential transactions | Sealevel, parallel transactions |
| Accounts | one address holds balance, nonce, code, storage | separate data accounts, programs are stateless |
| Token standard | ERC-20, ERC-721, ERC-1155 | SPL tokens |
| Addresses | 20-byte hex | base58, often via program-derived addresses |
| Main language | Solidity, Vyper | Rust, with the Anchor framework |

Both are decentralized compute. Neither runs the other's code, and anything moving between them crosses a bridge, which deserves its own audit.

## 12.3 Examples

Example 1: a two-test suite for the counter. This is the exact file that was run for this volume.

**TypeScript**

```typescript
import { test } from "node:test";
import assert from "node:assert/strict";
import hre from "hardhat";

test("increments", async function () {
  const { viem } = await hre.network.connect();
  const counter = await viem.deployContract("Counter");
  assert.equal(await counter.read.x(), 0n);
  await counter.write.inc();
  assert.equal(await counter.read.x(), 1n);
});

test("emits an event with the caller and the new value", async function () {
  const { viem } = await hre.network.connect();
  const publicClient = await viem.getPublicClient();
  const counter = await viem.deployContract("Counter");
  await counter.write.inc();
  const events = await publicClient.getContractEvents({
    address: counter.address,
    abi: counter.abi,
    eventName: "Incremented",
    fromBlock: 0n,
  });
  assert.equal(events.length, 1);
  assert.equal(events[0].args.by.toLowerCase(), "0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266");
  assert.equal(events[0].args.newValue, 1n);
});
```

**output**

```
  âœ” increments (3748ms)
  âœ” emits an event with the caller and the new value
2 passing (2 nodejs)
```

The event test reads through the public client, because the toolbox contract object exposes reads and writes, while event history comes from the client. The timing varies by machine; the counts are the meaningful part.

Example 2: the same suite in Foundry. Foundry is not installed in the environment used to verify this volume, so the command was NOT executed here.

**Solidity**

```
// test/Counter.t.sol - not executed in this environment
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test} from "forge-std/Test.sol";
import {Counter} from "../contracts/Counter.sol";

contract CounterTest is Test {
    Counter counter;

    function setUp() public {
        counter = new Counter();
    }

    function testIncrement() public {
        assertEq(counter.x(), 0);
        counter.inc();
        assertEq(counter.x(), 1);
    }
}
```

**shell**

```
forge test
```

Expected behaviour, in words: `forge test` compiles the test contract with the same compiler as the sources, runs `setUp` before every test function, and prints one line per test with a pass or fail marker, ending with a summary of passing and failing counts. `forge test -vvv` prints the execution trace of a failing case, which is where Foundry shines.

Example 3: verifying a deployed contract. The usage below is the real output of `npx hardhat verify --help` on the toolchain used for this volume; the command itself was not executed because it needs a live deployment and an explorer API key.

**shell**

```
npx hardhat verify --help
```

**output**

```
Verify a contract on all supported explorers

Usage: hardhat [GLOBAL OPTIONS] verify [--constructor-args-path <FILE_WITHOUT_DEFAULT>]
[--contract <STRING_WITHOUT_DEFAULT>] [--creation-tx-hash <STRING_WITHOUT_DEFAULT>]
[--force] [--libraries-path <FILE_WITHOUT_DEFAULT>] [--] address [constructorArgs]

OPTIONS

  --constructor-args-path      A relative path to a module that exports the constructor arguments
  --contract                   The name of the contract to verify, in the format <path>:<contractName>
  --creation-tx-hash           The transaction hash of the deployment
  --force                      Forces a new verification
  --libraries-path             A relative path to a module that exports any external libraries
```

**shell**

```
npx hardhat verify --network sepolia 0xYourContractAddress "10000000000000000000"
```

Expected behaviour, in words: the task asks the explorer to compile your sources with the recorded compiler settings and compare the bytecode; on success it prints confirmation and the explorer URL, and the contract page shows a read contract tab.

Example 4: a deploy script ready for a public network, reading the key from the environment. This was executed against a local node only.

**TypeScript**

```ts
import hre from "hardhat";

async function main() {
  const { viem } = await hre.network.connect();
  const supply = BigInt(process.env.INITIAL_SUPPLY ?? "1000");
  const token = await viem.deployContract("StudyToken", [supply]);
  console.log("StudyToken deployed to", token.address);
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

**output**

```
StudyToken deployed to 0x5fbdb2315678afecb367f032d93f642f64180aa3
```

On a public network the address differs, the call takes seconds instead of milliseconds, and `process.env.INITIAL_SUPPLY` keeps the supply out of the command line history.

## 12.4 Common mistakes and best practices

| Mistake | Why it hurts | Best practice |
|---|---|---|
| Testing only the happy path | exploits are not happy | one test per rule, including the reverts |
| Constructor arguments typed by hand | a silent mistyped parameter | export them from a module and read them in both places |
| Deploying from an interactive account | history and phishing exposure | a dedicated deployer with a hardware key |
| Deploying to a network by default | accidental mainnet sends | no default: name the network every time |
| Skipping verification | readers cannot audit the code | verify in the same script or the next command |
| No monitoring after launch | you hear about a bug from Twitter | alert on events, pauses, and ownership changes |

- Write the deploy script so it can re-run against a local network a hundred times without touching anything real.
- Record the transaction hash and the constructor arguments for every deployment; they are the inputs of the verification step.

## 12.5 Practice exercises

1. Why can an on-chain bug not be fixed the way a server bug is fixed?
2. What does a fork test need that a unit test does not?
3. Which command runs the test suite in Hardhat 3?
4. What is verified during block-explorer verification?
5. Name one safety net you should deploy together with a mainnet contract.

### Solutions

1. Because code is immutable once deployed: the fix is a new contract plus a migration, which is why proxy and upgrade patterns exist.
2. Access to an archive endpoint with the state of a real network, so calls against other protocols behave realistically.
3. `npx hardhat test`, run with Node's built-in test runner.
4. That the bytecode on-chain matches the bytecode the explorer produces by compiling the submitted source with the recorded settings.
5. A pause switch, a spending cap, or a timelock before privileged actions - any of them buys time when something is wrong.

## 12.6 Mini test

1. Multiple choice: which environment lets you interact with real protocol state without spending funds? - a) the in-process network - b) a fork test - c) a block explorer - d) a devnet faucet
2. True or false: deployed bytecode can be edited in place.
3. Multiple choice: which command verifies a deployment? - a) `npx hardhat verify` - b) `npx hardhat check` - c) `npx hardhat publish` - d) `npx hardhat audit`
4. Short answer: where should a deployer key come from?
5. True or false: Solana runs EVM bytecode natively.
6. Multiple choice: what is safe to store in the repository? - a) a deployer private key - b) constructor arguments and network configuration - c) a seed phrase - d) an explorer API key
7. Short code question: name the Hardhat command that deploys the counter contract to a running local node.

### Answer key and explanations

1. b - a fork test copies live state.
2. False - immutability is the point; behavior changes only through an upgrade path designed from the start.
3. a - `npx hardhat verify` with the address and constructor arguments.
4. From the environment or a hardware signer, never from the repository.

5. False - Solana has its own virtual machine and Rust programs.
6. b - configuration may live in the repository; secrets may not.
7. `npx hardhat run scripts/deploy.ts --network localhost`.

# Cheat sheet

## 13.1 Units and conversions

| Unit | Wei | Note |
|---|---|---|
| 1 wei | 1 | smallest unit |
| 1 gwei | 10^9 | fee prices are quoted in gwei |
| 1 ether | 10^18 | the display unit |

`ethers.parseEther("1.5")` gives `1500000000000000000n`; `viem`'s `parseEther` does the same. `ethers.formatUnits(x, 18)` and `formatEther(x)` go back.

## 13.2 Networks

| Network | Chain ID | Explorer |
|---|---|---|
| Ethereum mainnet | 1 | etherscan.io |
| Sepolia | 11155111 | sepolia.etherscan.io |
| Holesky | 17000 | holesky.etherscan.io |
| Local Hardhat node | 31337 | none, it is your machine |

## 13.3 Gas

| Term | Meaning |
|---|---|
| gas limit | ceiling the sender authorizes |
| gas used | what the execution actually consumed |
| base fee | burned, moves with demand |
| priority fee | tip to the validator |
| 21,000 | flat cost of a plain transfer |
| refund | the unused limit comes back |

Fee = gasUsed * (baseFee + priorityFee), in wei.

## 13.4 Solidity quick reference

| Concept | Syntax |
| --- | --- |
| file header | `// SPDX-License-Identifier: MIT` and `pragma solidity ^0.8.24;` |
| state variable | `uint256 public total;` |
| function | `function name(uint256 a) external view returns (uint256)` |
| payable | `function pay() external payable {}` |
| event | `event Sent(address indexed to, uint256 amount);` |
| emit | `emit Sent(msg.sender, msg.value);` |
| custom error | `error NotOwner(address caller);` then `revert NotOwner(msg.sender);` |
| require | `require(cond, "reason");` |
| modifier | `modifier onlyOwner() { require(msg.sender == owner); _; }` |
| mapping | `mapping(address => uint256) public balances;` |
| struct | `struct P { uint256 a; bool b; }` |
| array push | `items.push(item);` |
| visibility | `external`, `public`, `internal`, `private` |
| inheritance | `contract C is A { function f() public override(A) {} }` |
| time and sender | `block.timestamp`, `msg.sender`, `msg.value` |
| external call | `(bool ok, ) = to.call{value: amount}("");` |
| storage slot of a UUPS proxy | `0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc` |

## 13.5 Hashes and signatures

| Item | Value |
| --- | --- |
| curve | secp256k1 |
| address | last 20 bytes of keccak256(publicKey) |
| Transfer topic | `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef` |
| derivation path | m/44'/60'/0'/0/index |

## 13.6 Token standards

| Standard | Token | Key calls |
| --- | --- | --- |
| ERC-20 | fungible | `transfer`, `approve`, `transferFrom`, `balanceOf` |

| Standard | Token | Key calls |
|---|---|---|
| ERC-721 | unique | `ownerOf`, `safeTransferFrom`, `tokenURI`, `setApprovalForAll` |
| ERC-1155 | multi | `balanceOf(owner, id)`, `safeBatchTransferFrom`, `uri` |

## 13.7 OpenZeppelin imports

**Solidity**

```solidity
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC721} from "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import {ERC1155} from "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {UUPSUpgradeable} from "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
```

## 13.8 Clients

| Task | ethers 6 | viem |
|---|---|---|
| connect | `new ethers.JsonRpcProvider(url)` | `createPublicClient({ chain, transport: http() })` |
| chain id | `(await provider.getNetwork()).chainId` | `await client.getChainId()` |
| block | `provider.getBlockNumber()` | `client.getBlockNumber()` |
| balance | `provider.getBalance(addr)` | `client.getBalance({ address })` |
| sign | `new ethers.Wallet(key, provider)` | `privateKeyToAccount(key)` + `createWalletClient` |
| deploy | `ContractFactory.deploy()` | `walletClient.deployContract({ abi, bytecode })` |
| call | `contract.f(args)` | `client.readContract({ address, abi, functionName, args })` |
| send | `contract.f(args)` | `walletClient.writeContract({ address, abi, functionName, args })` |
| wait | `tx.wait()` | `client.waitForTransactionReceipt({ hash })` |
| events | `contract.queryFilter("Ev", 0, "latest")` | `client.getContractEvents({ address, abi, eventName, fromBlock })` |
| units | `parseEther`, `formatEther` | `parseEther`, `formatEther` |

## 13.9 Hardhat and Foundry

| Action | Hardhat 3 | Foundry |
|---|---|---|
| new project | npm install, config | `forge init` |

| Action | Hardhat 3 | Foundry |
|---|---|---|
| compile | `npx hardhat compile` | `forge build` |
| test | `npx hardhat test` | `forge test` |
| deploy script | `npx hardhat run scripts/x.ts --network localhost` | `forge create C --rpc-url ... --private-key ...` |
| local chain | `npx hardhat node` | `anvil` |
| verify | `npx hardhat verify` | `forge verify-contract` |
| CLI query | node script | `cast call` / `cast send` |

## 13.10 The DAO runbook

| Step | Command or action |
|---|---|
| 1 | propose, event `Proposed` |
| 2 | vote once per address, event `Voted` |
| 3 | wait past `deadline`, state becomes `Passed` |
| 4 | execute, event `Executed` |

# Final test

1. Multiple choice: what cryptographic property locks blocks into a chain? - a) each block is encrypted with the network key - b) each block includes the hash of the previous block - c) each block is signed by the whole network - d) each block has a timestamp
2. True or false: an address is the last 20 bytes of the keccak256 hash of the public key.
3. Multiple choice: which network has chain ID 31337? - a) Sepolia - b) mainnet - c) a local Hardhat node - d) Holesky
4. Short answer: how much gas does a plain ether transfer consume?
5. Multiple choice: which mutability marker allows a function to receive ETH? - a) `view` - b) `pure` - c) `payable` - d) `external`
6. True or false: on solc 0.8, `x - 1` reverts when `x` is zero.
7. Multiple choice: which storage location is cheapest for a read-only function parameter? - a) `storage` - b) `memory` - c) `calldata` - d) the stack
8. Short code question: write the one-line modifier that only lets the owner pass.
9. Multiple choice: which file does Hardhat 3 read its configuration from? - a) `hardhat.config.js` - b) `hardhat.config.ts` - c) `package.json` - d) `truffle.js`
10. True or false: `npx hardhat test` in Hardhat 3 runs mocha.
11. Multiple choice: which token standard supports batch transfers? - a) ERC-20 - b) ERC-721 - c) ERC-1155 - d) ERC-165

12. Short answer: what event pair replaces a from or to address with the zero address?
13. Multiple choice: which method does a browser wallet expose for account requests? - a) `eth_accounts` - b) `eth_requestAccounts` - c) `personal_unlockAccount` - d) `wallet_connect`
14. True or false: `eth_call` can change chain state.
15. Multiple choice: what is `topics[0]` of an event log? - a) the block hash - b) the contract address - c) the keccak256 hash of the event signature - d) the sender address
16. Short answer: which ordering of steps defeats reentrancy?
17. Multiple choice: which modifier blocks a second entry into a function during one call? - a) `onlyOwner` - b) `nonReentrant` - c) `whenNotPaused` - d) `initializer`
18. True or false: a proxy contract holds the state while the implementation holds the code.
19. Multiple choice: which formula prices an automated market maker? - a) x + y = k - b) x * y = k - c) x - y = k - d) x / y = k
20. Short answer: what does a timelock delay?
21. Multiple choice: which assertion belongs in a test that checks an event was emitted? - a) the receipt's gas used - b) the number and arguments of the decoded logs - c) the deployer address - d) the block number only
22. True or false: verifying a contract on a block explorer proves the deployed bytecode matches the submitted source.
23. Multiple choice: which family of contract avoids unbounded loops over user-extendable storage? - a) contracts that always iterate everything - b) contracts with caps and pagination - c) contracts that read only the latest block - d) contracts without mappings
24. Short answer: name two things a local chain prints that a public network does not.
25. Multiple choice: which ordering is the safe transaction lifecycle? - a) send, simulate, wait - b) simulate, send, wait for the receipt - c) wait, send, simulate - d) simulate only

## 14.1 Final test answers

1. b - the parent hash binds the history together.
2. True - 20 bytes, displayed with an EIP-55 checksum.
3. c - `npx hardhat node` and `anvil` both use 31337.
4. 21,000 gas, a flat protocol cost for a value transfer.
5. c - `payable`, the only marker that accepts value.
6. True - checked arithmetic reverts on underflow by default.
7. c - `calldata` is the read-only arguments area and is cheapest.
8. `modifier onlyOwner() { require(msg.sender == owner); _; }`
9. b - Hardhat 3 projects are typed and the config is TypeScript.
10. False - Hardhat 3 uses Node's built-in `node:test` runner.
11. c - ERC-1155 defines batch operations over many ids.
12. A mint from the zero address and a burn to it.
13. b - `eth_requestAccounts`, which the wallet renders as a prompt.

14. False - a call is simulated locally and writes nothing.
15. c - the signature hash, identical for every contract emitting it.
16. Checks, then effects, then interactions.
17. b - `nonReentrant`, from OpenZeppelin's `ReentrancyGuard`.
18. True - joined by `delegatecall` through the ERC-1967 slot.
19. b - the constant product invariant, X times Y equals k.
20. Execution of a passed governance proposal, so holders can exit or react.
21. b - the test must assert the decoded logs and their arguments, not just a receipt.
22. True - that is exactly what the explorer's compile-and-compare proves.
23. b - caps plus pagination keep the gas cost bounded.
24. Development private keys and prefunded balances; both are worthless on real networks.
25. b - simulate first so a revert costs nothing, then send and wait.

# Sources

Every page below was consulted while preparing this volume, and the examples were executed against the documentation of the tools named in the running notes.

Ethereum documentation:


- https://ethereum.org/en/web3/
- https://ethereum.org/en/whitepaper/
- https://ethereum.org/en/developers/docs/intro-to-ethereum/
- https://ethereum.org/en/developers/docs/accounts/
- https://ethereum.org/en/developers/docs/transactions/
- https://ethereum.org/en/developers/docs/gas/
- https://ethereum.org/en/developers/docs/evm/
- https://ethereum.org/en/developers/docs/networks/
- https://ethereum.org/en/developers/docs/nodes-and-clients/
- https://ethereum.org/en/developers/docs/apis/json-rpc/
- https://ethereum.org/en/developers/docs/standards/tokens/
- https://ethereum.org/en/developers/docs/standards/tokens/erc-20/
- https://ethereum.org/en/developers/docs/standards/tokens/erc-721/
- https://ethereum.org/en/developers/docs/standards/tokens/erc-1155/
- https://ethereum.org/en/developers/docs/smart-contracts/
- https://ethereum.org/en/developers/docs/smart-contracts/security/
- https://ethereum.org/en/developers/docs/smart-contracts/upgrading/
- https://ethereum.org/en/defi/
- https://ethereum.org/en/dao/

Solidity documentation:

- https://docs.soliditylang.org/
- https://docs.soliditylang.org/en/latest/types.html
- https://docs.soliditylang.org/en/latest/control-structures.html
- https://docs.soliditylang.org/en/latest/contracts.html
- https://docs.soliditylang.org/en/latest/abi-spec.html

Tooling:

- https://hardhat.org/tutorial
- https://hardhat.org/docs
- https://book.getfoundry.sh/
- https://docs.ethers.org/v6/
- https://viem.sh/docs

Data and storage:

- https://thegraph.com/docs/
- https://docs.ipfs.tech/
- https://docs.ipfs.tech/concepts/content-addressing/

Standards referenced by number:

- https://eips.ethereum.org/EIPS/eip-20
- https://eips.ethereum.org/EIPS/eip-155
- https://eips.ethereum.org/EIPS/eip-721
- https://eips.ethereum.org/EIPS/eip-1155
- https://eips.ethereum.org/EIPS/eip-1559
- https://eips.ethereum.org/EIPS/eip-1967
- https://eips.ethereum.org/EIPS/eip-1193

Reference implementations:

- https://docs.openzeppelin.com/contracts/

Other ecosystems:

- https://solana.com/docs
- https://www.anchor-lang.com/